INTELLIGENT EVENT DATA RECORDER FOR VEHICLES

A Project Report submitted to

Jawaharlal Nehru Technological University

in partial fulfillment of the requirements for the award of Degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

K. SAI KRISHNA **Roll No.:22831A0452**

Under the Guidance of

Mr M.RAMESH NAIK

(Assistant professor)

Department of Electronics and Communication Engineering

Guru Nanak Institute of Technology

Ibrahimpatnam, Hyderabad, R.R. District501506

May,2025

www.gnithyd.ac.in

CERTIFICATE

This is to certify that the Mini-project entitled "INTELLIGENT EVENT DATA RECORDER FOR VEHICLES" is being submitted by K SAI KRISHNA bearing Roll No. 22831A0452 in partial fulfillment for the award of the Degree of Bachelor of the Technology in ELECTRONICS AND COMMUNICATION ENGINEERING to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by her them under my guidance and supervision.

The results embodied in this Mini-project report have not been submitted to any other University or Institute for the award of any Degree or Diploma

Internal Guide

Mr.M. Ramesh naik

Assistant professor

Project coordinator

Mr.D. Naresh kumar

Assistant professor

Head of the Department

Dr.S.P.Yadav

HOD&Dean Academics

External Examiner

DECLARATION OF STUDENT

I, K SAI KRISHNA (22831A0452) hereby declare that the major project titled "INTELLIGENT EVENT DATA RECORDER FOR VEHICLES" has been carried out by me as part of the requirements for the award of the Degree of B-TECH in the Department of ECE at Guru Nanak Institute of Technology.

I/We confirm the following:

1. The project was undertaken by us under the supervision of our guide, Mr.M. Ramesh naik, from the selection of the topic to the completion of the final report.
2. I/We have ensured that the results presented in the report are accurate and based on our original work.
3. To the best of our knowledge, the content of this report is free from plagiarism and adheres to ethical standards.
4. Each member of the team has contributed significantly and appropriately to the project work.
5. The project report has been prepared with diligence, ensuring clarity, accuracy, and adherence to academic standards.

I/We further declare that this report has not been submitted, in part or full, to any other institution or university for the award of any degree or diploma.

K Sai Krishna Signature

22831A0452

Date:

Place:

DECLARATION OF GUIDE

I, Mr.M. Ramesh naik, hereby declare that I have guided the major project titled “Intelligent Event Data Recorder for vehicles” undertaken by K Sai Krishna (22831A0452). This project was carried out towards the fulfillment of the requirements for the award of the Degree of B-TECH in ECE at Guru Nanak Institute of Technology.

As the guide, I confirm the following:

1. I have overseen the entire project process, from the selection of the project title to the submission of the final report.
2. I have reviewed and certified the accuracy and relevance of the results presented in the report.
3. The work carried out is original, free from plagiarism, and adheres to ethical guidelines.
4. The contributions of each student have been appropriately recognized and assessed.
5. The project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of Guide: Mr. M. Ramesh naik signature of the Guide

Date:

Place:Ibrahimpatnam ,Hyderabad

Name of HOD: Dr.S.P.Yadav signature of the HOD

Department: ECE

Date:

Place: Ibrahimpatnam ,Hyderabad

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our internal guide, **Mr.M.Ramesh Naik**, Associate Professor, Electronics and Communication Engineering, for his valuable guidance, encouragement, and continuous support throughout the duration of this project.

We are also thankful to **Dr.S.P.Yadav**, HOD and Dean Academics, Electronics and Communication Engineering, for his expert supervision and helpful suggestions, which contributed significantly to the successful completion of this project.

We would like to express my profound sense of gratitude to **Dr. K. Venkata Rao**, **Principa**l, for his constant and valuable guidance.

We would like to thank **Dr. Sanjeev Shrivastava, Director,** for his valuable support

We would like to express our deep sense of gratitude to **Dr. H. S. Saini, Managing Director**, Guru Nanak Group of Institutions for his tremendous support, encouragement, and inspiration. Lastly, we thank the almighty, our parents, and friends for their constant encouragement without which this assignment would not be possible. We would like to thank all the other staff members, both teaching and non-teaching, which have extended their timely help and eased my work.

We would also like to thank the faculty members of the **Electronics and Communication Engineering** and the Lab Technicians for their assistance and cooperation during the practical work of our project.

We are grateful to our friends and well-wishers for their encouragement, collaboration, and useful feedback throughout the project journey. Lastly, we sincerely thank our parents for their constant support, patience, and motivation, which helped us complete this project successfully.

K Sai Krishna **22831A0452**

TABLE OF CONTENTS

ABSTRACT

The main purpose of this wireless black box project is to develop a vehicle black box system that can be installed into any vehicle all over the world. This paradigm is often designed with minimum range of circuits. Wireless black box is basically a device that will indicate all the parameters of a vehicle crash and will also store and display its parameters at every three seconds such as date, time, temperature, location, vibration, alcohol limit etc. At the time of accident or abnormal states of sensors, the message will be sent from the system built inside the car to the registered mobile numbers such as emergency numbers of police stations, hospitals, family members, owner etc. We have used various types of sensors like temperature sensor, Vibration sensor measures vibrations felt by the car during accident.

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

1.1GENERAL

One of the fastest developing areas in the domain of autonomous robotics and artificial intelligence (AI) with a potential for a large scale deployment in short or medium term is self-driving technologies and smart mobility systems based on them. The already started transition of conventional transport systems and infrastructure to collaborative intelligent transportation systems based on self-driving technologies, ubiquitous connectivity and traffic management with minimal human intervention promises the first integration of advanced AI into social life of such magnitude. Given many uncertainties inherent not only in technology developments but also in user acceptance, societal expectations and problems, changes needed to legal and regulatory systems and many more, it is instrumental for the success of the transition to ensure that the societal expectations and policies are embedded early in the technology development process. For addressing the challenge of integrating the "application pull" determined by social needs, expectations and problems with the "technology push" originating from the accelerating development of technologies and specific solutions, we have developed the Policy Scan and Technology Strategy Design methodology. The methodology is conceived for identifying and negotiating concrete societal problems and technological solutions that mitigate them by facilitating a dialog between social world and the world of technological availabilities in immediate contexts and situations. For the in-depth presentation and discussion of the Policy Scan and Technology Strategy Design methodology see [18], which is a predecessor to the current article and is based on the results of the same research project. In a nutshell, the method described in [18] is a design inquiry encompassing three stages which are iteratively visited many times until concrete societal problem and a technological solution mitigating it are identified and negotiated between involved parties. These stages are: (1) Content analysis (of document and interviews), (2) Identifying / correcting directions of inquiry, (3) Modeling the solution. The goal of the methodology is to provide a systemic approach to consolidate huge informational context into socially acceptable and sustainable innovation road-map for both companies and governments. The current paper describes the outcome of the Policy Scan and Strategy Design process in a chosen application domain of autonomous driving and smart mobility – which resulted in identification of in-vehicle data recording, storage and access management technology. The identified technology is a concrete

junction point between the world of current societal problems and expectations in the domain and the technology world providing availabilities to mitigate these problems. As discussed in [18], European Union is among the most progressive governments which start to draft proposals of how to approach legal, regulatory and governance aspects related to the introduction of advanced autonomous robots and AI technologies into society (considering, among other aspects, disruption of the job market, ethical issues and the possibility that in the long term AIs will surpass human intellectual capacity). One of the ways to integrate AI to society is to pave a way for understanding independent decisions of AI through equipping all autonomous robots with a "black box" enabling recording, storing and analyzing their internal operations, logic and decisions 1 . In case of autonomous vehicles, this requirement relates closely to the upcoming legislation requiring to upgrade the Event Data Recording devices (which are currently being installed in most cars) accommodating them to much richer and technically challenging automated driving context. Such devices would help reconstructing traffic events, understand causes of behavior of autonomous cars and autonomous traffic managers, solve legal, insurance and technical issues. Yet it also implies collecting, storing and sharing huge amounts of data at least part of which is highly privacy sensitive and regulated by international and EU data privacy and provenance laws. In-vehicle data recording, storage and access management is a close to the market technological innovation witch addresses important legal challenges inherent in implementing intelligent transportation systems. Apart from the immediate function, this technology provides a stepping stone towards implementation of transparency principle of autonomous robots and AI in the long term. Last but not least, development and implementation of in-vehicle data recording, storage and access management systems is a business opportunity for entering dynamic and quickly growing market. The article is structured as follows. In Section 2 we explain major developments of autonomous driving industry, relevant governance bodies and legislative domains. The section introduces relations between social and technology worlds in the autonomous driving and smart mobility domain. Section 3 addresses the legislative and technological developments, the debate of industry stakeholders around the in-vehicle data access solutions and their place in collaborative intelligent transportation systems (C-ITS). Subsection 3.3 presents in-depth analysis of a proposed concrete technological solution for solving specific societal problem having clear market deployment potential – Event Data Recorder for Autonomous Driving (EDR-AD). Section 4 concludes the article and outlines further work.

1.2EXISTING SYSTEM:

In existing system there is no such system allocated in the vehicle manual surveillance will be held as either accident is recorded in the CCTV surveillance and intimated to the emergency services.

1.3 PROPOSED SYSTEM

The main purpose of this paper is to develop a prototype of the Vehicle Black Box System VBBS that can be installed into any vehicle all over the world. This prototype can be designed with minimum number of circuits. The VBBS can contribute to constructing safer vehicles, improving the treatment of crash victims, helping insurance companies with their vehicle crash investigations, and enhancing road status in order to decrease the death rate.

CHAPTER 2

LITERATURE SURVEY

1. Title: Driver Distraction Detection Using Event Cameras**.**

Authors: Waseem Shariff and team.

Year: 2024.

Focus: They used a special camera and brain like AI to spot when drivers are distracted.

Research Gap: It works well but it needs more testing in real life driving and different weather or lighting**.**

2. Title: Testing EDRs Without Crashing a Vehicle.

Authors: Matthew DiSogra and team.

Year: 2024.

Focus: They made away to test EDRs by simulating vehicle speed instead of crashig cars.

Research Gap: Only simulates speed doesn't include all the sensors a real crash would use.

3. Title: Seatbelt Detection System.

Authors: Paul Kielty and team**.**

Year: 2023.

Focus: They created a system that checks if you are wearig a seatbelt using a small smart camera.

Research Gap: Might not work the same in all the cars or lighting conditions**.**

4. Title: Better Tracking with Event Cameras**.**

Authors: Sumin Hu and team.

Year: 2022.

Focus: They made a system that tracks moving things quickly, use ful for cars that record events**.**

Research Gap: Needs to be tested in busy or complex driving situation

CHAPTER 3

DESIGN AND DEVELOPMENT

3.1NEED ANALYSIS

An embedded system can be defined as a computing device that does a specific focused job. Appliances such as the air-conditioner, VCD player, DVD player, printer, fax machine, mobile phone etc. are examples of embedded systems. Each of these appliances will have a processor and special hardware to meet the specific requirement of the application along with the embedded software that is executed by the processor for meeting that specific requirement. The embedded software is also called “firm ware”. The desktop/laptop computer is a general-purpose computer. You can use it for a variety of applications such as playing games, *word* processing, accounting, software development and so on. In contrast, the software in the embedded systems is always fixed listed below:

· Embedded systems do a very specific task; they cannot be programmed to do different things.

. Embedded systems have very limited resources, particularly the memory. Generally, they do not have secondary storage devices such as the CDROM or the floppy disk. Embedded systems have to work against some deadlines. A specific job has to be completed within a specific time. In some embedded systems, called real-time systems, the deadlines are stringent. Missing a deadline may cause a catastrophe-loss of life or damage to property. Embedded systems are constrained for power. As many embedded systems operate through a battery, the power consumption has to be very low.

· Some embedded systems have to operate in extreme environmental conditions such as very high temperatures and humidity.

Application Areas

Nearly 99 per cent of the processors manufactured end up in embedded systems. The embedded system market is one of the highest growth areas as these systems are used in very market segment- consumer electronics, office automation, industrial automation, biomedical engineering, wireless communication, data communication, telecommunications, transportation, military and so on.

Consumer appliances:

At home we use a number of embedded systems which include digital camera, digital diary, DVD player, electronic toys, microwave oven, remote controls for TV and air-conditioner, VCO player, video game consoles, video recorders etc. Today's high-tech car has about 20 embedded systems for transmission control, engine spark control, air-conditioning, navigation etc. Even wristwatches are now becoming embedded systems. The palmtops are powerful embedded systems using which we can carry out many general-purpose tasks such as playing games and word processing.

Office Automation:

The office automation products using embedded systems are copying machine, fax machine, key telephone, modem, printer, scanner etc.

Industrial Automation:

Today a lot of industries use embedded systems for process control. These include pharmaceutical, cement, sugar, oil exploration, nuclear energy, electricity generation and transmission. The embedded systems for industrial use are designed to carry out specific tasks such as monitoring the temperature, pressure, humidity, voltage, current etc., and then take appropriate action based on the monitored levels to control other devices or to send information to a centralized monitoring station. In hazardous industrial environment, where human presence has to be avoided, robots are used, which are programmed to do specific jobs. The robots are now becoming very powerful and carry out many interesting and complicated tasks such as hardware assembly.

Medical Electronics:

Almost every medical equipment in the hospital is an embedded system. These equipment's include diagnostic aids such as ECG, EEG, blood pressure measuring devices, X-ray scanners; equipment used in blood analysis, radiation, colonoscopy, endoscopy etc. Developments in medical electronics have paved way for more accurate diagnosis of diseases.

Computer Networking:

Computer networking products such as bridges, routers, Integrated Services Digital Networks (ISDN), Asynchronous Transfer Mode (ATM), X.25 and frame relay switches are embedded systems which implement the necessary data communication protocols. For example, a router interconnects two networks. The two networks may be running different protocol stacks. The router's function is to obtain the data packets from incoming pores, analyze the packets and send them towards the destination after doing necessary protocol conversion. Most networking equipments, other than the end systems (desktop computers) we use to access the networks, are embedded systems.

Telecommunications:

In the field of telecommunications, the embedded systems can be categorized as subscriber terminals and network equipment. The subscriber terminals such as key telephones, ISDN phones, terminal adapters, web cameras are embedded systems. The network equipment includes multiplexers, multiple access systems, Packet Assemblers Dissemblers (PADs), sate11ite modems etc. IP phone, IP gateway, IP gatekeeper etc. are the latest embedded systems that provide very low-cost voice communication over the Internet.

Wireless Technologies:

Advances in mobile communications are paving way for many interesting applications using embedded systems. The mobile phone is one of the marvels of the last decade of the 20'h century. It is a very powerful embedded system that provides voice communication while we are on the move. The Personal Digital Assistants and the palmtops can now be used to access multimedia service over the Internet. Mobile communication infrastructure such as base station controllers, mobile switching centres are also powerful embedded systems.

Insemination:

Testing and measurement are the fundamental requirements in all scientific and engineering activities. The measuring equipment we use in laboratories to measure parameters such as weight, temperature, pressure, humidity, voltage, current etc. are all embedded systems. Test equipment such as oscilloscope, spectrum analyzer, logic analyzer, protocol analyzer, radio

communication test set etc. are embedded systems built around powerful processors. Thank to miniaturization, the test and measuring equipment are now becoming portable facilitating easy testing and measurement in the field by field-personnel.

Security:

Security of persons and information has always been a major issue. We need to protect our homes and offices; and also the information we transmit and store. Developing embedded systems for security applications is one of the most lucrative businesses nowadays. Security devices at homes, offices, airports etc. for authentication and verification are embedded systems. Encryption devices are nearly 99 per cent of the processors that are manufactured end up in~ embedded systems. Embedded systems find applications in every industrial segment- consumer electronics, transportation, avionics, biomedical engineering, manufacturing, process control and industrial automation, data communication, telecommunication, defense, security etc. Used to encrypt the data/voice being transmitted on communication links such as telephone lines. Biometric systems using fingerprint and face recognition are now being extensively used for user authentication in banking applications as well as for access control in high security buildings.

Finance:

Financial dealing through cash and cheques are now slowly paving way for transactions using smart cards and ATM (Automatic Teller Machine, also expanded as Any Time Money) machines. Smart card, of the size of a credit card, has a small micro-controller and memory; and it interacts with the smart card reader! ATM machine and acts as an electronic wallet. Smart card technology has the capability of ushering in a cashless society. Well, the list goes on. It is no exaggeration to say that eyes wherever you go, you can see, or at least feel, the work of an embedded system.

3.2SYSTEM ARCHITECTURE

Overview of Embedded System Architecture

Every embedded system consists of custom-built hardware built around a Central Processing Unit (CPU). This hardware also contains memory chips onto which the software is loaded. The software residing on the memory chip is also called the 'firmware'. The embedded system architecture can be represented as a layered architecture as shown in Fig. The operating system runs above the hardware, and the application software runs above the operating system. The same architecture is applicable to any computer including a desktop computer. However, there are significant differences. It is not compulsory to have an operating system in every embedded system. For small appliances such as remote control units, air conditioners, toys etc., there is no need *for* an operating system and you can write only the software specific to that application. For applications involving complex processing, it is advisable to have an operating system. In such a case, you need to integrate the application software with the operating system and then transfer the entire software on to the memory chip. Once the software is transferred to the memory chip, the software will continue to run *for* a long time you don't need to reload new software.

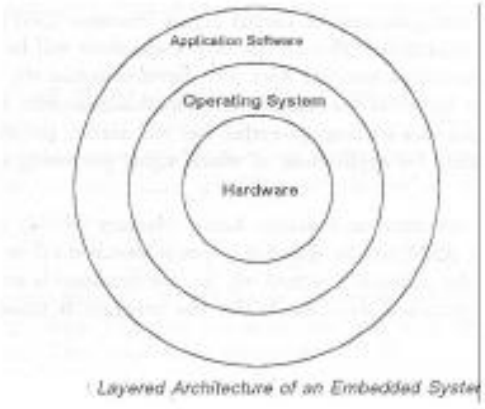


Fig 3.2:System Architecture

Now, let us see the details of the various building blocks of the hardware of an embedded system. As shown in Fig. the building blocks are;

· Central Processing Unit (CPU)

· Memory (Read-only Memory and Random Access Memory)

· Input Devices

· Output devices

· Communication interfaces

· Application-specific circuitry

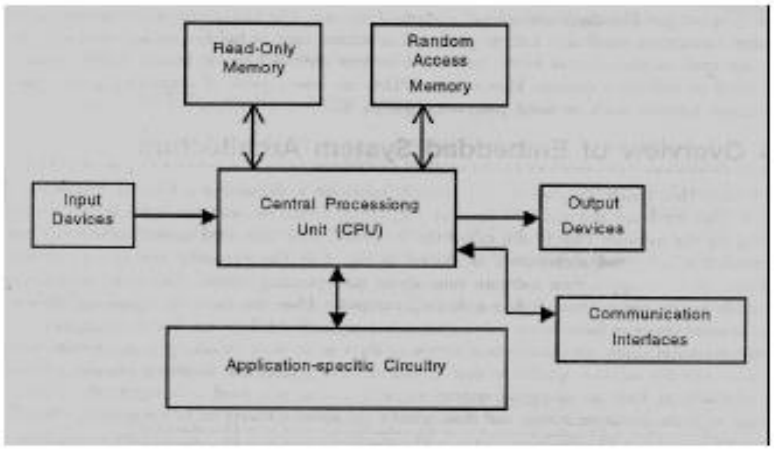


Fig:Flow Chart

Central Processing Unit (CPU):

The Central Processing Unit (processor, in short) can be any of the following: microcontroller, microprocessor or Digital Signal Processor (DSP). A micro-controller is a low-cost processor. Its main attraction is that on the chip itself, there will be many other components such as memory, serial communication interface, analog-to digital converter etc. So, for small applications, a micro-controller is the best choice as the number of external components required will be very less. On the other hand, microprocessors are more powerful, but you need to use many external components with them. D5P is used mainly for applications in which signal processing is involved such as audio and video processing.

Memory:

The memory is categorized as Random Access Memory (RAM) and Read Only Memory (ROM). The contents of the RAM will be erased if power is switched off to the chip, whereas ROM retains the contents even if the power is switched off. So, the firmware is stored in the ROM. When power is switched on, the processor reads the ROM; the program is program is executed.

Input Devices:

Unlike the desktops, the input devices to an embedded system have very limited capability. There will be no keyboard or a mouse, and hence interacting with the embedded system is no easy task. Many embedded systems will have a small keypad-you press one key to give a

specific command. A keypad may be used to input only the digits. Many embedded systems used in process control do not have any input device *for* user interaction; they take inputs *from* sensors or transducers 1'fnd produce electrical signals that are in turn fed to other systems.

Output Devices:

The output devices of the embedded systems also have very limited capability. Some embedded systems will have a *few* Light Emitting Diodes (LEDs) *to* indicate the health status of the system modules, or *for* visual indication of alarms. A small Liquid Crystal Display (LCD) may also be used to display *some* important parameters.

Communication Interfaces:

The embedded systems may need to, interact with other embedded systems at they may have to transmit data to a desktop. To facilitate this, the embedded systems are provided with one or a *few* communication interfaces such as RS232, RS422, RS485, Universal Serial Bus (USB), IEEE 1394, Ethernet etc.

Application-Specific Circuitry:

Sensors, transducers, special processing and control circuitry may be required fat an embedded system, depending on its application. This circuitry interacts with the processor to carry out the necessary work. The entire hardware has to be given power supply either through the 230 volts main supply or through a battery. The hardware has to design in such a way that the power consumption is minimized.

3.2.1BLOCK DIAGRAM:

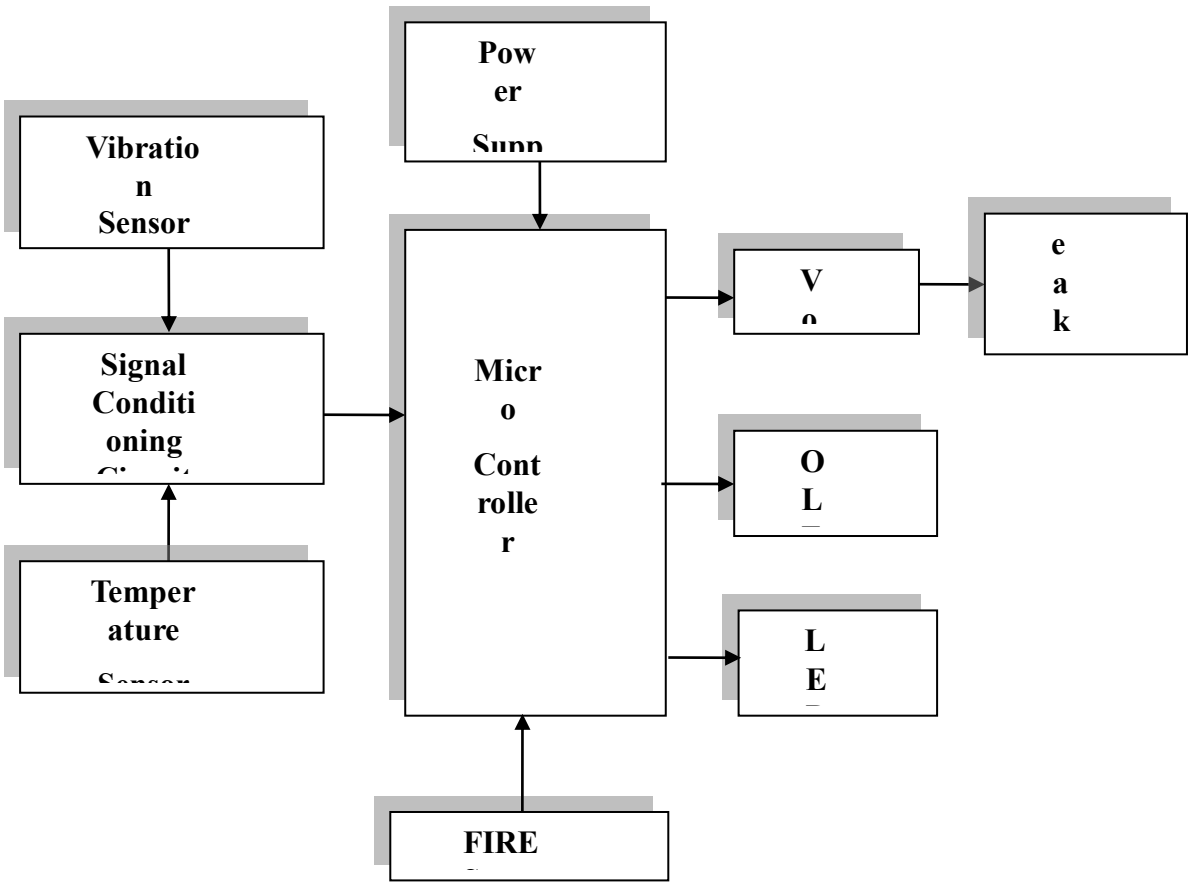


Fig 3.2.1:Block Diagram

3.2.2 POWER SUPPLY

The power supply section is the section which provide +5V for the components to work. IC LM7805 is used for providing a constant power of +5V.

The ac voltage, typically 220V, is connected to a transformer, which steps down that ac voltage down to the level of the desired dc output. A diode rectifier then provides a full-wave rectified voltage that is initially filtered by a simple capacitor filter to produce a dc voltage. This resulting dc voltage usually has some ripple or ac voltage variation.

A regulator circuit removes the ripples and also retains the same dc value even if the input dc voltage varies, or the load connected to the output dc voltage changes. This voltage regulation is usually obtained using one of the popular voltage regulator IC units.

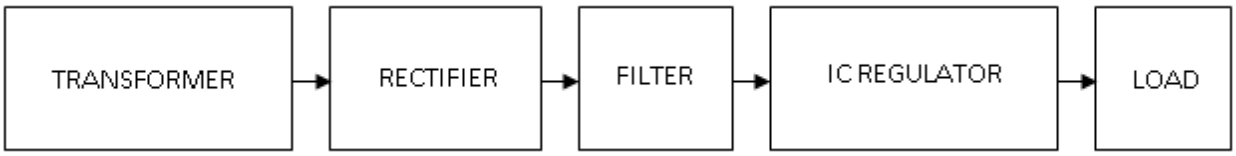


Fig 3.2.2: Block Diagram Of Power Supply

3.2.3 Transformer

Transformers convert AC electricity from one voltage to another with little loss of power. Transformers work only with AC and this is one of the reasons why mains electricity is AC.

Step-up transformers increase voltage, step-down transformers reduce voltage. Most power supplies use a step-down transformer to reduce the dangerously high mains voltage (230V in India) to a safer low voltage.

The input coil is called the primary and the output coil is called the secondary. There is no electrical connection between the two coils; instead they are linked by an alternating magnetic field created in the soft-iron core of the transformer. Transformers waste very little power so the power out is (almost) equal to the power in. Note that as voltage is stepped down current is stepped up.

The transformer will step down the power supply voltage (0-230V) to (0- 6V) level. Then the secondary of the potential transformer will be connected to the bridge rectifier, which is constructed with the help of PN junction diodes. The advantages of using bridge rectifier are it will give peak voltage output as DC.

3.2.4 Rectifier

There are several ways of connecting diodes to make a rectifier to convert AC to DC. The bridge rectifier is the most important and it produces full-wave varying DC. A full-wave rectifier can also be made from just two diodes if a centre-tap transformer is used, but this method is rarely used now that diodes are cheaper. A single diode can be used as a rectifier but it only uses the positive (+) parts of the AC wave to produce half-wave varying Dc

3.2.5Bridge Rectifier

When four diodes are connected as shown in figure, the circuit is called as bridge rectifier. The input to the circuit is applied to the diagonally opposite corners of the network, and the output is taken from the remaining two corners. Let us assume that the transformer is working properly and there is a positive potential, at point A and a negative potential at point B. the positive potential at point A will forward bias D3 and reverse bias D4.

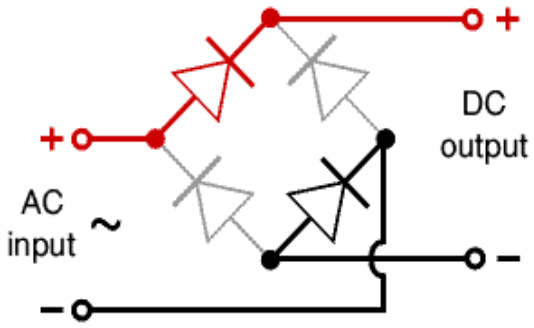


Fig 3.2.5:Bridge Rectifier

The negative potential at point B will forward bias D1 and reverse D2. At this time D3 and D1 are forward biased and will allow current flow to pass through them; D4 and D2 are reverse biased and will block current flow.

One advantage of a bridge rectifier over a conventional full-wave rectifier is that with a given transformer the bridge rectifier produces a voltage output that is nearly twice that of the conventional full-wave circuit.

i. The main advantage of this bridge circuit is that it does not require a special centre tapped transformer, thereby reducing its size and cost.

ii. The single secondary winding is connected to one side of the diode bridge network and the load to the other side as shown below.

iii. The result is still a pulsating direct current but with double the frequency.

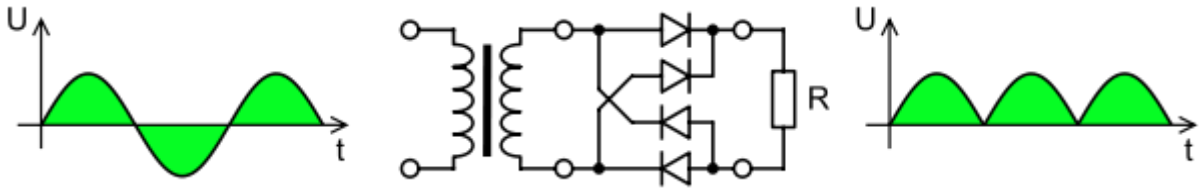


Fig:3.2.5.1:Output Waveform Of DC

Smoothing

Smoothing is performed by a large value electrolytic capacitor connected across the DC supply to act as a reservoir, supplying current to the output when the varying DC voltage from the rectifier is falling. The capacitor charges quickly near the peak of the varying DC, and then discharges as it supplies current to the output.

3.2.6Voltage Regulators

Voltage regulators comprise a class of widely used ICs. Regulator IC units contain the circuitry for reference source, comparator amplifier, control device, and overload protection all in a single IC. IC units provide regulation of either a fixed positive voltage, a fixed negative voltage, or an adjustably set voltage. The regulators can be selected for operation with load currents from hundreds of milli amperes to tens of amperes, corresponding to power ratings from milli watts totens of watts.

A fixed three-terminal voltage regulator has an unregulated dc input voltage, Vi, applied to one input terminal, a regulated dc output voltage, Vo, from a second terminal, with the third terminal connected to ground.

The series 78 regulators provide fixed positive regulated voltages from 5 to 24 volts. Similarly, the series 79 regulators provide fixed negative regulated voltages from 5 to 24 volts. Voltage regulator ICs are available with fixed (typically 5, 12 and 15V) or variable output voltages. They are also rated by the maximum current they can pass. Negative voltage regulators are available, mainly for use in dual supplies. Most regulators include some automatic protection from excessive current ('overload protection') and overheating ('thermal protection').

Many of the fixed voltage regulator ICs has 3 leads and look like power transistors, such as the 7805 +5V 1Amp regulator. They include a hole for attaching a heat sink if necessary.

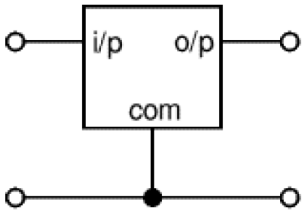


Fig3.2.6:Regulator

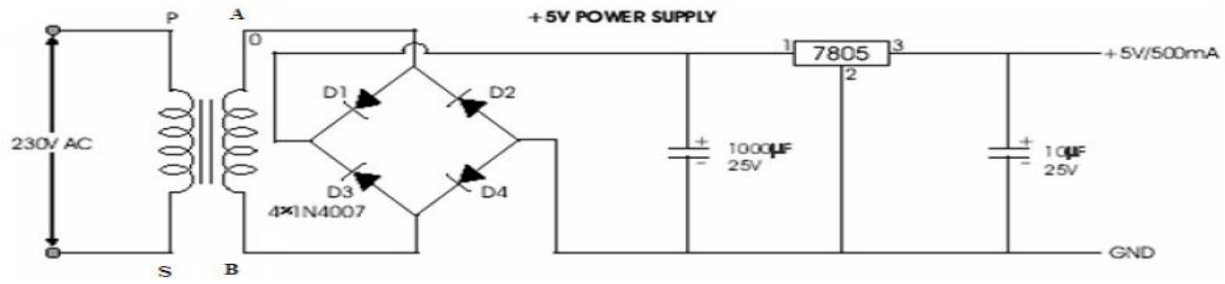


Fig 3.2.6.1:Circuit Diagram Of Power Supply

3.3DESIGN METHODOLOGY

3.3.1BLOCK DIAGRAM:

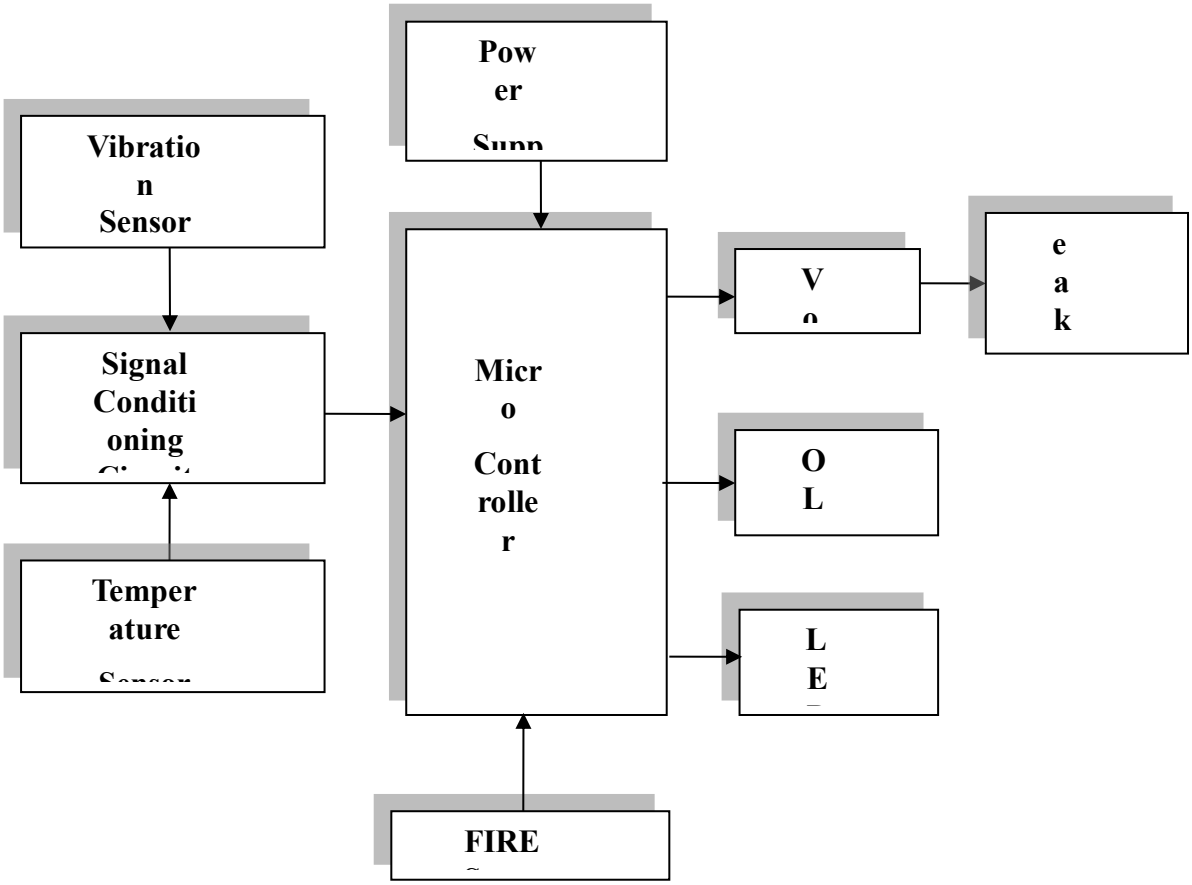


Fig 3.3.1:Block Diagram

This block diagram outlines the components of a system, likely a safety or monitoring device, that uses various sensors to detect environmental conditions and then processes and outputs information.

Here's an explanation of each block:

- **Power Supply:** This is the essential component that provides the necessary electrical energy to operate all other parts of the system. It converts available power (e.g., from batteries or an AC adapter) into the stable DC voltages required by the electronic components.
- **Micro Controller:** This is the "brain" of the system. It's a small, integrated circuit designed to control specific functions. In this context, the microcontroller will:

* Read data from all the connected sensors.

* Process and interpret the sensor data (e.g., determine if a threshold has been crossed for vibration, fire, or temperature).

* Control the output devices (OLED, Voice IC).

* Implement the system's logic and decision-making.

- **Vibration Sensor:** This sensor detects physical vibrations or tremors. It could be used to sense impacts, movement, or unusual shaking, which might indicate an anomaly or a dangerous situation.
- **Signal Conditioning Circuit:** Sensors often output analog signals that are weak or noisy. This circuit takes the raw output from the vibration sensor (and potentially other analog sensors if they are not directly compatible with the microcontroller's ADC) and processes it. This processing might include:

  * **Amplification:** Boosting the signal strength.

  * **Filtering:** Removing unwanted noise.

  * Conversion: Converting analog signals to digital (if the microcontroller doesn't have a suitable built-in ADC or if specific resolution/speed is needed).
- **FIRE Sensor:** This sensor is designed to detect the presence of fire. This could be a flame sensor (detects the infrared/ultraviolet light emitted by flames) or a smoke sensor (detects smoke particles).
- **Temperature Sensor:** This sensor measures the ambient temperature. It could be used to detect abnormally high temperatures, which might indicate a fire, overheating equipment, or other critical conditions.
- **OLED (Organic Light-Emitting Diode):** This is a display device. It will be used by the microcontroller to visually output information to the user. This could include:

  * Sensor readings (e.g., current temperature, vibration level).

  * Status messages (e.g., "Normal," "Alert!").

  * Warnings (e.g., "Fire Detected!").
- **Voice IC (Integrated Circuit):** This component is responsible for generating pre-recorded or synthesized voice messages. The microcontroller would trigger the Voice IC to play specific audio alerts or instructions to the user. This adds an audible warning or informational capability to the system.

  Overall Function (Inferred):

Based on these components, the system appears to be designed for environmental monitoring, specifically focusing on safety and alerting. It continuously monitors for:

* Vibrations: Possibly for intrusion detection, structural integrity monitoring, or equipment malfunction.

* Fire: For early fire detection and warning.

* Temperature: For detecting abnormal heating, which could be a precursor to fire or indicate a problem.

Upon detecting any anomalies (e.g., fire, high temperature, significant vibration), the microcontroller would then activate the OLED to display visual information and the Voice IC to provide audible alerts, ensuring that users are promptly informed of the detected conditions.

3.4COMPONENT DESIGN

MICROCONTROLLER

3.4.1ARDUINO UNO

Arduino/genuino uno is a microcontroller board based on the atmega328p (datasheet). It has 14 digital input/output pins (of which 6 can be used as pwm outputs), 6 analog inputs, a 16 MHz quartz crystal, a usb connection, a power jack, an icsp header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a usb cable or power it with a ac-to-dc adapter or battery to get started.. You can tinker with your uno without worrying too much about doing something wrong, worst case scenario you can replace the chip for a few dollars and start over again.

"Uno" means one in italian and was chosen to mark the release of arduino software (ide) 1.0. The uno board and version 1.0 of arduino software (ide) were the reference versions of arduino, now evolved to newer releases.

Fig3.4.1:Arduino uno

3.4.2OLED (Organic Light Emitting Diodes)

OLED (Organic Light Emitting Diodes) is a flat light emitting technology, made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. OLEDs are emissive displays that do not require a backlight and so are thinner and more efficient than LCD displays (which do require a white backlight).

OLED displays are not just thin and efficient - they provide the best image quality ever and they can also be made transparent, flexible, foldable and even rollable and stretchable in the future. OLEDs represent the future of display technology!

OLED vs LCD

An OLED display have the following advantages over an LCD display:

- Improved image quality - better contrast, higher brightness, fuller viewing angle, a wider color range and much faster refresh rates.
- Lower power consumption.
- Simpler design that enables ultra-thin, flexible, foldable and transparent displays
- Better durability - OLEDs are very durable and can operate in a broader temperature range

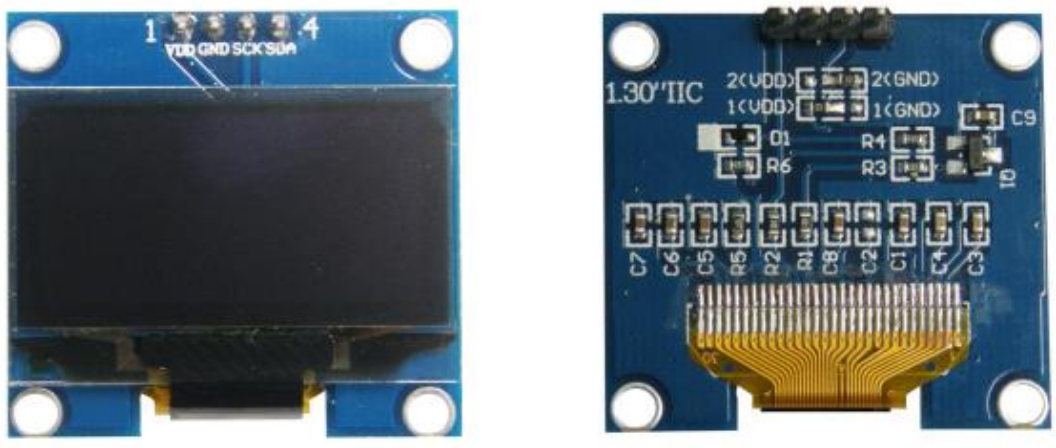


Fig3.4.2:OLED

The future - flexible and transparent OLED displays

As we said, OLEDs can be used to create flexible and transparent displays. This is pretty exciting as it opens up a whole world of possibilities:

- Curved OLED displays, placed on non-flat surfaces
- Wearable OLEDs
- Foldable OLEDs and rollable OLEDs which can be used to create new mobile devices
- Transparent OLEDs embedded in windows or car windshdields
- And many more we cannot even imagine today...

Flexible OLEDs are already on the market for many years (in smartphones, wearables and other devices) and since 2019, with the introduction of the Samsung Galaxy Fold, foldable devices are increasing in popularity. In 2019 LG also announced the world's first rollable OLED - its 65" OLED R TV that can roll into its base!

An OLED is made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. Click here for a more detailed view of the OLED technology.

OLEDs are organic because they are made from carbon and hydrogen. There's no connection to organic food or farming - although OLEDs are very efficient and do not contain any bad metals - so it's a real green technology.

OLED is the best display technology - and indeed OLED panels are used today to create the most stunning TVs ever - with the best image quality combined with the thinnest sets ever. And this is only the beginning, as in the future OLED will enable large rollable and transparent TVs!

Currently the only company that produces OLED TV panels is LG Display. The Korean display maker is producing a wide range of OLED TV panels, offering these to LG Electronics, Panasonic, Sony, Philips and others.

OLED white lighting

OLEDs can be used to create excellent light source. OLEDs offer diffuse area lighting and can be flexible, efficient, light, thin, transparent, color-tunable and more. OLEDs enable new designs and these devices emit healthier light compared to CFLs and LED lighting devices.

Specifications

- ✓ Use CHIP No.SH1106
- ✓ Use 3.3V-5V POWER SUPPLY
- ✓ Graphic LCD 1.3" in width with 128x64 Dot Resolution
- ✓ White Display is used for the model OLED 1.3 I2C WHITE and blue Display is used for the model OLED 1.3 I2C BLUE
- ✓ Use I2C Interface
- ✓ Directly connect signal to Microcontroller 3.3V and 5V without connecting through Voltage Regulator Circuit
- ✓ Total Current when running together is 8 mA - PCB Size: 33.7 mm x 35.5 mm

Table3.4.2.1:Table shows name and function of Pin OLED

Pin No.	Pin Name	Description
1	VDD	Pin Power Supply for LCD, using 3.3V-5V
2	GND	Pin Ground
3	SCK	Pin SCL of I2C Interface
4	SDA	Pin SDA of I2C Interface

Example of connecting with Board Arduino This example illustrates how to connect together with Board Arduino, in this case, it is Board ET-BASE AVR EASY328. It is used together with Program Arduino and Library to connect and communicate to Module OLED. - Firstly, install Library "u8glib"; go to Menu Sketch > Include Library > Add.ZIP Library...

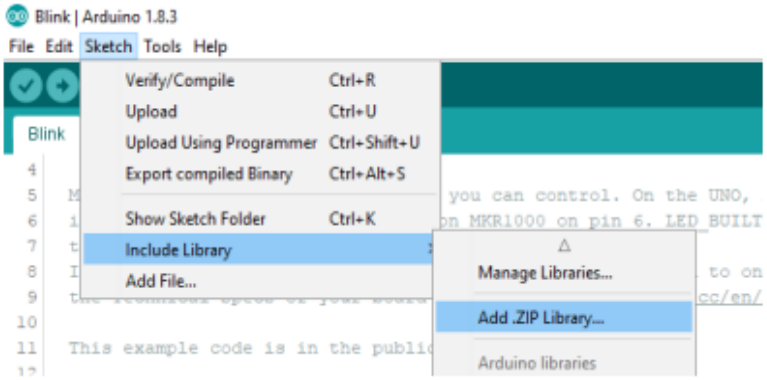


Fig3.4.2.2:Sketch Box

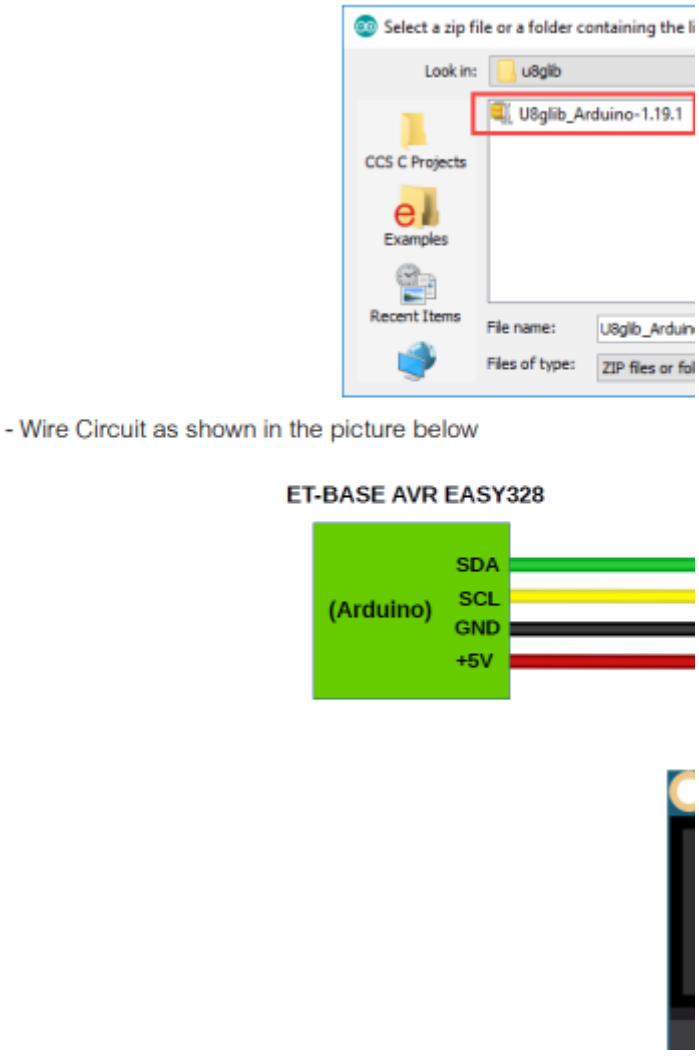


Fig3.4.2.2: Go to Folder Lib Arduino\u8glib in CD-ROM; next, choose hown in the below.

3.4.3VIBRATION SENSOR

Sensor is mounted at the bottom of the unit. The unit should be fixed with the vibrating body firmly the sensitivity is adjusted for the required vibration/ shock is detected the output goes low and the delay is provided for proper operation vibrating frequency and amplitude can be detected.

SPEC:

Supply- DC +2v ripple free

Output current-PNP 100ma

Analog o/p- 10ma

Sensors available to detect the flame / fire smoke flow level speed position Temp Bio medical application.

Special sensor can be developed against specific requirement.

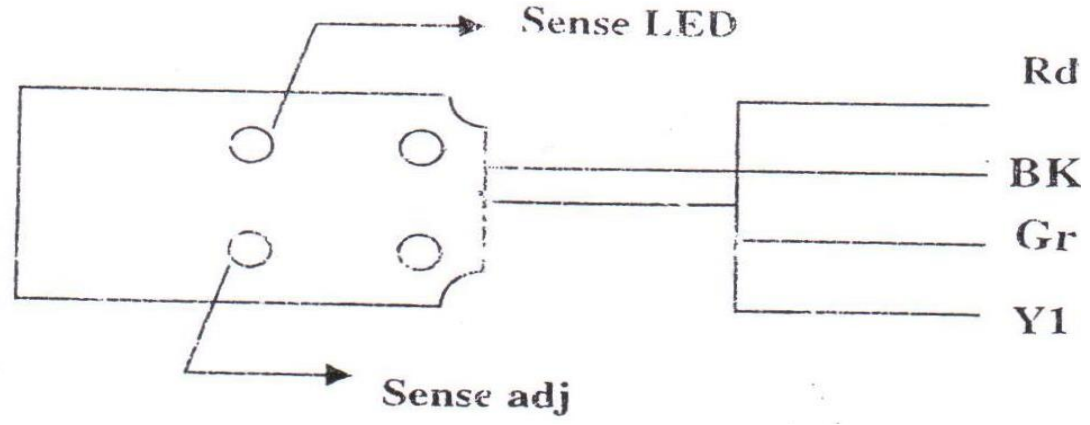


Fig 3.4.3:Vibration Sensor

Rd- 12v

BK- 0v

Gr- switch output

Y1- analog output vibration

Table3.4.4 :DHT11 – TEMPERATURE AND HUMIDITY SENSOR

Pin Identification and Configuration:

No:	Pin Name	Description
For DHT11 Sensor		
1	Vcc	Power supply 3.5V to 5.5V
2	Data	Outputs both Temperature and Humidity through serial Data
3	NC	No Connection and hence not used
4	Ground	Connected to the ground of the circuit
For DHT11 Sensor module		
1	Vcc	Power supply 3.5V to 5.5V
2	Data	Outputs both Temperature and Humidity through serial Data
3	Ground	Connected to the ground of the circuit

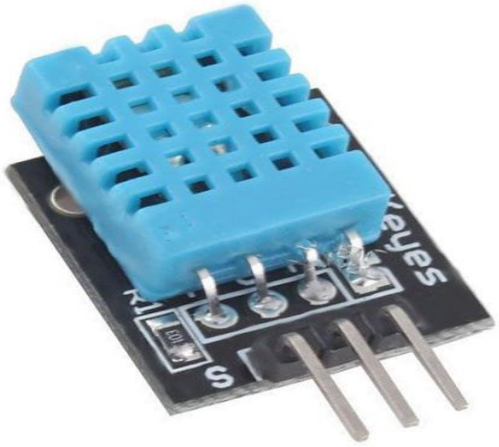


Fig3.4.4.1:DHT11 Sensor

DHT11 Specifications:

- Operating Voltage: 3.5V to 5.5V
- Operating current: 0.3mA (measuring) 60uA (standby)
- Output: Serial data
- Temperature Range: 0°C to 50°C
- Humidity Range: 20% to 90%
- Resolution: Temperature and Humidity both are 16-bit
- Accuracy: ±1°C and ±1%

The **DHT11 sensor** can either be purchased as a sensor or as a module. Either way, the performance of the sensor is same. The sensor will come as a 4-pin package out of which only three pins will be used whereas the module will come with three pins as shown above.
The only difference between the sensor and module is that the module will have a filtering capacitor and pull-up resistor inbuilt, and for the sensor, you have to use them externally if required.

The **DHT11** is a commonly used **Temperature and humidity sensor.** The sensor comes with a dedicated NTC to measure temperature and an 8-bit microcontroller to output the values of temperature and humidity as serial data. The sensor is also factory calibrated and hence easy to interface with other microcontrollers.
The sensor can measure temperature from 0°C to 50°C and humidity from 20% to 90% with an accuracy of ±1°C and ±1%. So if you are looking to measure in this range then this sensor might be the right choice for you.

The DHT11 Sensor is factory calibrated and outputs serial data and hence it is highly easy to set it up. The connection diagram for this sensor is shown below.

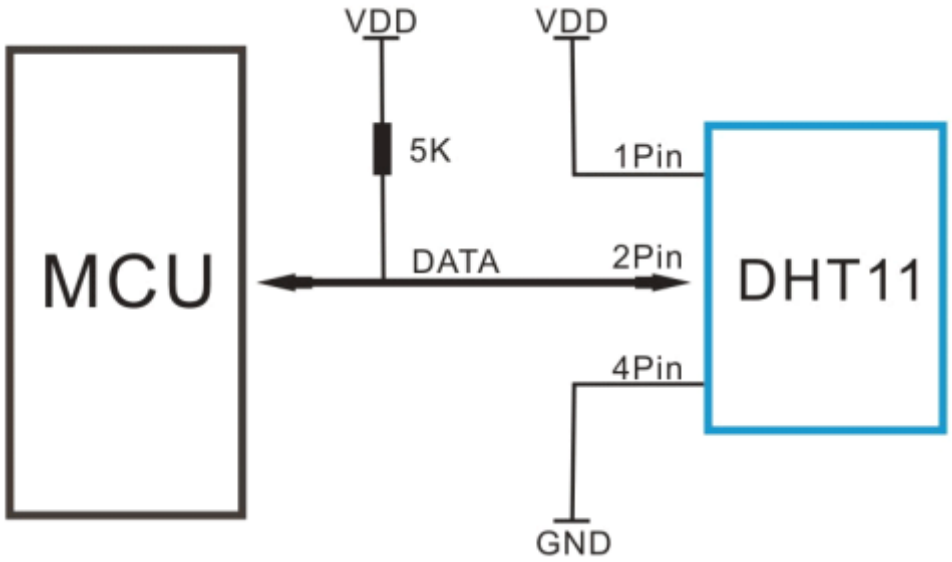


Fig3.4.4.2:Block Diagram of DHT11

As you can see the data pin is connected to an I/O pin of the MCU and a 5K pull-up resistor is used. This data pin outputs the value of both temperature and humidity as serial data. If you

are trying to interface DHT11 with Arduino then there are ready-made libraries for it which will give you a quick start.

If you are trying to interface it with some other MCU then the datasheet given below will come in handy. The output given out by the data pin will be in the order of 8bit humidity integer data + 8bit the Humidity decimal data +8 bit temperature integer data + 8bit fractional temperature data +8 bit parity bit.

3.4.5 FIRE SENSOR

The Fire sensor, as the name suggests, is used as a simple and compact device for protection against fire. The module makes use of IR sensor and comparator to detect fire up to a range of 1-2 meters.

The device, weighing about 5 grams, can be easily mounted on the device body. It gives a high output on detecting fire. This output can then be used to take the requisite action. An on-board LED is also provided for visual indication.

Feature

- Typical Maximum Range :2 m .
- Indicator LED with 3 pin easy interface connector.
- Operating Voltage 5v

An automatic fire sensor sense the unwanted presence of fire by monitoring environmental changes associated with combustion. In general, a fire alert system is classified as either automatically actuated, manually actuated, or both. Fire sensor are intended to notify the fire in the building occupants to evacuate in the event of a fire or other emergency, report the event to an off-premises location in order to summon emergency services and associated

Fig3.4.5:Fire sensor

- Fire alarm control panel: This component, the hub of the system, monitors inputs and system integrity, controls outputs and relays information.
- Primary Power supply: Commonly the non-switched 120 or 240 Volt Alternating Current source supplied from a commercial power utility. In non-residential applications, a branch

circuit is dedicated to the fire alarm system and its constituents. "Dedicated branch circuits" should not be confused with "Individual branch circuits" which supply energy to a single appliance.

- Secondary (backup) Power supplies: This component, commonly consisting of sealed lead-acid storage batteries or other emergency sources including generators, is used to supply energy in the event of a primary power failure.
- Initiating Devices: This component acts as an input to the fire alarm control unit and are either manually or automatically actuated. Examples would be devices like pull stations or smoke detectors.
- Notification appliances: This component uses energy supplied from the fire alarm system or other stored energy source, to inform the proximate persons of the need to take action, usually to evacuate. This is done by means of a flashing light, strobe light, electromechanical horn, speaker, or a combination of these devices.
- Building Safety Interfaces: This interface allows the fire alarm system to control aspects of the built environment and to prepare the building for fire and to control the spread of smoke fumes and fire by influencing air movement, lighting, process control, human transport and exit.

Manually actuated devices; Break glass stations, Buttons and manual fire alarm activation are constructed to be readily located (near the exits), identified, and operated.

Automatically actuated devices can take many forms intended to respond to any number of detectable physical changes associated with fire: convicted thermal energy; heat detector, products of combustion; smoke detector, radiant energy; flame detector, combustion gasses; carbon monoxide detector and release of extinguishing agents; water-flow detector. The newest innovations can use cameras and computer algorithms to analyze the visible effects of fire and movement in applications inappropriate for or hostile to other detection methods.

3.4.6 BUZZER

A buzzer or beeper is a signaling device, usually electronic, typically used in automobiles, house hold appliances such as a microwave oven, or game shows.

It most commonly consists of a number of switches or sensors connected to a control unit that determines if and which button was pushed or a preset time has lapsed, and usually illuminates a light on the appropriate button or control panel, and sounds a warning in the form of a continuous or intermittent buzzing or beeping sound. Initially this device was based on an electromechanical system which was identical to an electric bell without the metal gong (which makes the ringing noise). Often these units were anchored to a wall or ceiling and used the ceiling or wall as a sounding board. Another implementation with some AC-connected devices was to implement a circuit to make the AC current into a noise loud enough to drive a loudspeaker and hook this circuit up to a cheap 8-ohm speaker. Nowadays, it is more popular to use a ceramic-based piezoelectric sounder like a Sonalert which makes a high-pitched tone. Usually these were hooked up to "driver" circuits which varied the pitch of the sound or pulsed the sound on and off.

In game shows it is also known as a "lockout system," because when one person signals ("buzzes in"), all others are locked out from signalling. Several game shows have large buzzer buttons which are identified as "plungers".

Fig3.4.6: Buzzer

USES

- Annunciator panels
- Electronic metronomes
- Game shows
- Microwave ovens and other household appliances
- Sporting events such as basketball games
- Electrical alarms

Table 3.4.7:TECHNICAL SPECIFICATIONS

Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Analog Input Pins	6
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Clock Speed	16 MHz
LED_BUILTIN	13
Length	68.6 mm
Width	53.4 mm
Weight	25 g

3.5TOOLS AND TECHNOLOGIES USED

3.5.1 PROGRAMMING

The arduino/genuino uno can be programmed with the (arduino software (ide)). Select "arduino/genuino uno from the tools > board menu (according to the microcontroller on your board). For details, see the reference and tutorials.

The atmega328 on the arduino/genuino uno comes pre-programmed with a bootloader that allows you to upload new code to it without the use of an external hardware programmer. It communicates using the original stk500 protocol (reference, c header files).

You can also bypass the bootloader and program the microcontroller through the icsp (in-circuit serial programming) header using arduino isp or similar; see these instructions for details.

The atmega16u2 (or 8u2 in the rev1 and rev2 boards) firmware source code is available in the arduino repository. The atmega16u2/8u2 is loaded with a dfu bootloader, which can be activated by:

On rev1 boards: connecting the solder jumper on the back of the board (near the map of italy) and then reseting the 8u2.

On rev2 or later boards: there is a resistor that pulling the 8u2/16u2 hwb line to ground, making it easier to put into dfu mode.

You can then use atmel's flip software (windows) or the dfu programmer (mac os x and linux) to load a new firmware. Or you can use the isp header with an external programmer (overwriting the dfu bootloader). See this user-contributed tutorial for more information.

WARNINGS

The arduino/genuino uno has a resettable polyfuse that protects your computer's usb ports from shorts and overcurrent. Although most computers provide their own internal protection, the fuse provides an extra layer of protection. If more than 500 ma is applied to the usb port, the fuse will automatically break the connection until the short or overload is removed.

DIFFERENCES WITH OTHER BOARDS

The uno differs from all preceding boards in that it does not use the ftdi usb-to-serial driver chip. Instead, it features the atmega16u2 (atmega8u2 up to version r2) programmed as a usb-to-serial converter.

POWER

The arduino/genuino uno board can be powered via the usb connection or with an external power supply. The power source is selected automatically.

External (non-usb) power can come either from an ac-to-dc adapter (wall-wart) or battery. The adapter can be connected by plugging a 2.1mm center-positive plug into the board's power jack. Leads from a battery can be inserted in the gnd and vin pin headers of the power connector.

The board can operate on an external supply from 6 to 20 volts. If supplied with less than 7v, however, the 5v pin may supply less than five volts and the board may become unstable. If using more than 12v, the voltage regulator may overheat and damage the board. The recommended range is 7 to 12 volts.

The power pins are as follows:

Vin. The input voltage to the arduino/genuino board when it's using an external power source (as opposed to 5 volts from the usb connection or other regulated power source). You can supply voltage through this pin, or, if supplying voltage via the power jack, access it through this pin.

5v.this pin outputs a regulated 5v from the regulator on the board. The board can be supplied with power either from the dc power jack (7 - 12v), the usb connector (5v), or the vin pin of the board (7-12v). Supplying voltage via the 5v or 3.3v pins bypasses the regulator, and can damage your board. We don't advise it.

3v3. A 3.3 volt supply generated by the on-board regulator. Maximum current draw is 50 ma.

Gnd. Ground pins.

Ioref. This pin on the arduino/genuino board provides the voltage reference with which the microcontroller operates. A properly configured shield can read the ioref pin voltage and select the appropriate power source or enable voltage translators on the outputs to work with the 5v or 3.3v.

MEMORY

The atmega328 has 32 kb (with 0.5 kb occupied by the bootloader). It also has 2 kb of sram and 1 kb of eeprom (which can be read and written with the eeprom library).

Input and output

See the mapping between arduino pins and atmega328p ports. The mapping for the atmega8, 168, and 328 is identical.

SOFTWARE SPECIFICATIONS

3.5.2 THE ARDUINO INTEGRATED DEVELOPMENT ENVIRONMENT

Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuino hardware to upload programs and communicate with them.

WRITING SKETCHES

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension. ino. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom righthand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

NB: Versions of the Arduino Software (IDE) prior to 1.0 saved sketches with the extension .pde. It is possible to open these files with version 1.0, you will be prompted to save the sketch with the .ino extension on save.

Verify

Checks your code for errors compiling it.

Upload

Compiles your code and uploads it to the configured board. See uploading below for details.

Note: If you are using an external programmer with your board, you can hold down the "shift" key on your computer when using this icon. The text will change to "Upload using Programmer"

New

Creates a new sketch.

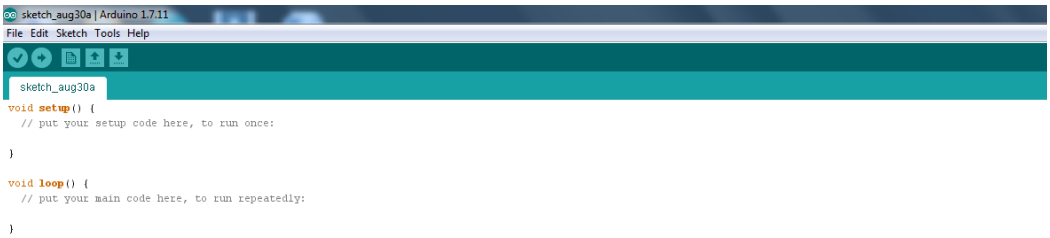


Fig 3.5.2.1 Creates a new sketch.

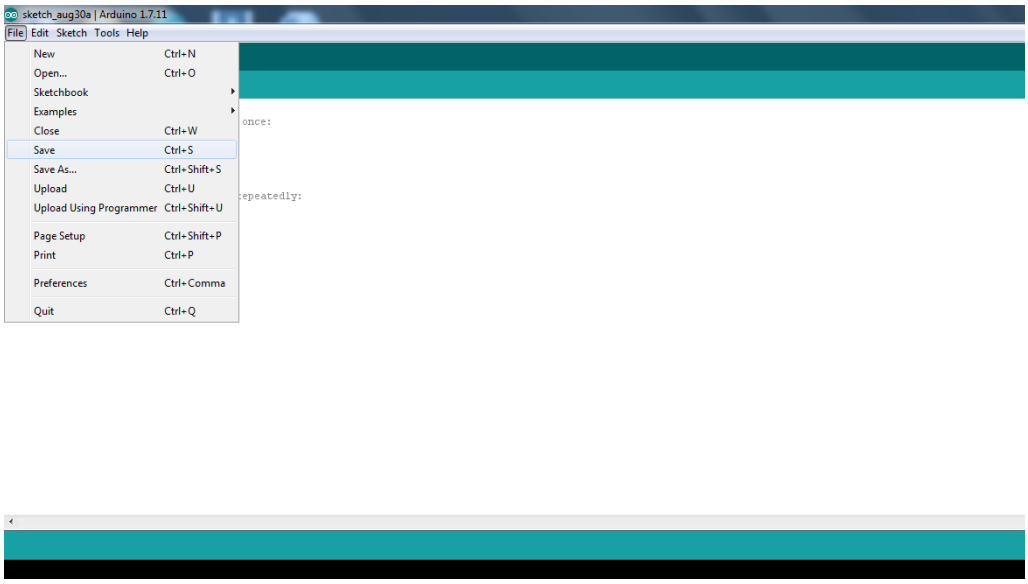
Open

Presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window overwriting its content.

Note: due to a bug in Java, this menu doesn't scroll; if you need to open a sketch late in the list, use the File | Sketchbookmenu instead.

Save

Saves your sketch.



SerialMonitor

fig:3.5.2.2: Opens the serial monitor.

Additional commands are found within the five menus: File, Edit, Sketch, Tools, Help. The menus are context sensitive, which means only those items relevant to the work currently being carried out are available.

FILE

- *New*

  Creates a new instance of the editor, with the bare minimum structure of a sketch already in place.
- *Open*

  Allows to load a sketch file browsing through the computer drives and folders.
- *OpenRecent*

  Provides a short list of the most recent sketches, ready to be opened.

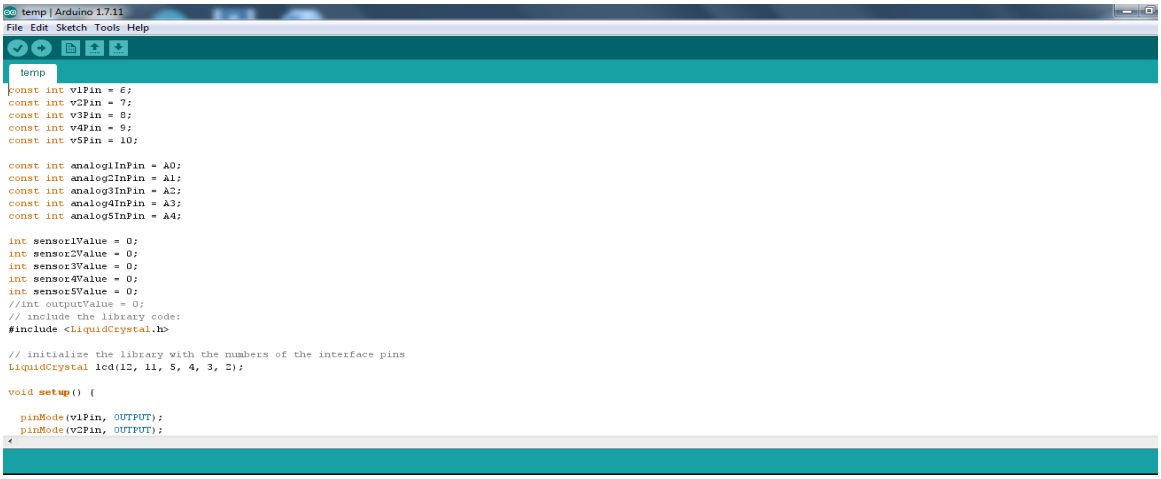


Fig 3.5.2.3: Open Recent Code

- *Sketchbook*

  Shows the current sketches within the sketchbook folder structure; clicking on any name opens the corresponding sketch in a new editor instance.

- *Examples*

  Any example provided by the Arduino Software (IDE) or library shows up in this menu item. All the examples are structured in a tree that allows easy access by topic or library.

- *Close*

  Closes the instance of the Arduino Software from which it is clicked.

- *Save*

  Saves the sketch with the current name. If the file hasn't been named before, a name will be provided in a "Save as.." window.

- *Saveas...*

  Allows to save the current sketch with a different name.

- *PageSetup*

  It shows the Page Setup window for printing.

- *Print*

  Sends the current sketch to the printer according to the settings defined in Page Setup.

- *Preferences*

  Opens the Preferences window where some settings of the IDE may be customized, as the language of the IDE interface.

- *Quit*

  Closes all IDE windows. The same sketches open when Quit was chosen will be automatically reopened the next time you start the IDE.

EDIT

- *Undo/Redo*

  Goes back of one or more steps you did while editing; when you go back, you may go forward with Redo.
- *Cut*

  Removes the selected text from the editor and places it into the clipboard.
- *Copy*

  Duplicates the selected text in the editor and places it into the clipboard.
- *Copy for Forum*

  Copies the code of your sketch to the clipboard in a form suitable for posting to the forum, complete with syntax coloring.
- *Copy as HTML*

  Copies the code of your sketch to the clipboard as HTML, suitable for embedding in web pages.
- *Paste*

  Puts the contents of the clipboard at the cursor position, in the editor.
- *Select All*

  Selects and highlights the whole content of the editor.
- *Comment/Uncomment*

  Puts or removes the // comment marker at the beginning of each selected line.
- *Increase/Decrease Indent*

  Adds or subtracts a space at the beginning of each selected line, moving the text one space on the right or eliminating a space at the beginning.
- *Find*

  Opens the Find and Replace window where you can specify text to search inside the current sketch according to several options.
- *Find Next*

  Highlights the next occurrence - if any - of the string specified as the search item in the Find window, relative to the cursor position.
- *Find Previous*

  Highlights the previous occurrence - if any - of the string specified as the search item in the Find window relative to the cursor position.

SKETCH

- *Verify/Compile*

  Checks your sketch for errors compiling it; it will report memory usage for code and variables in the console area.

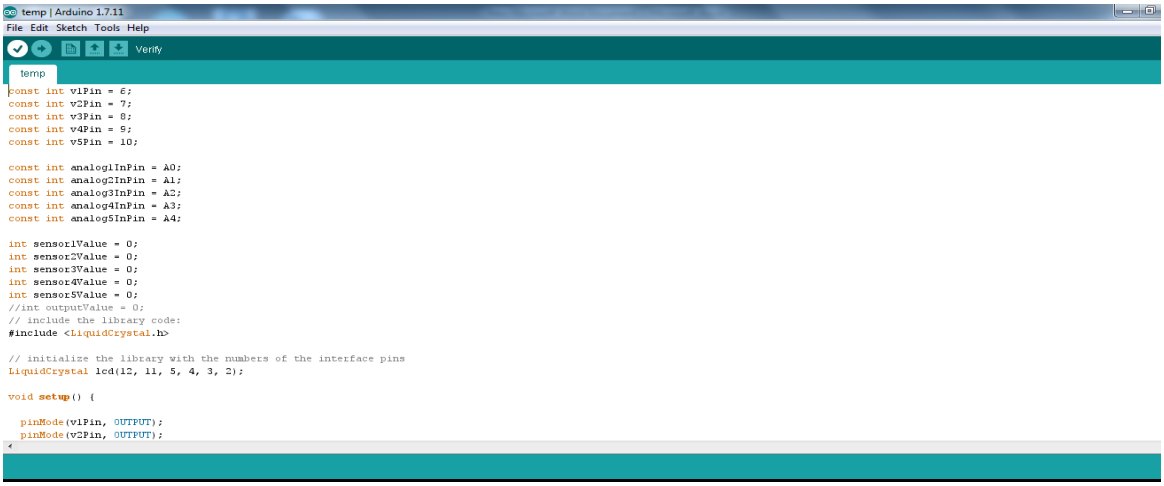


Fig:3.5.2.4: verify and compile the code

- *Upload* -

  Compiles and loads the binary file onto the configured board through the configured Port.

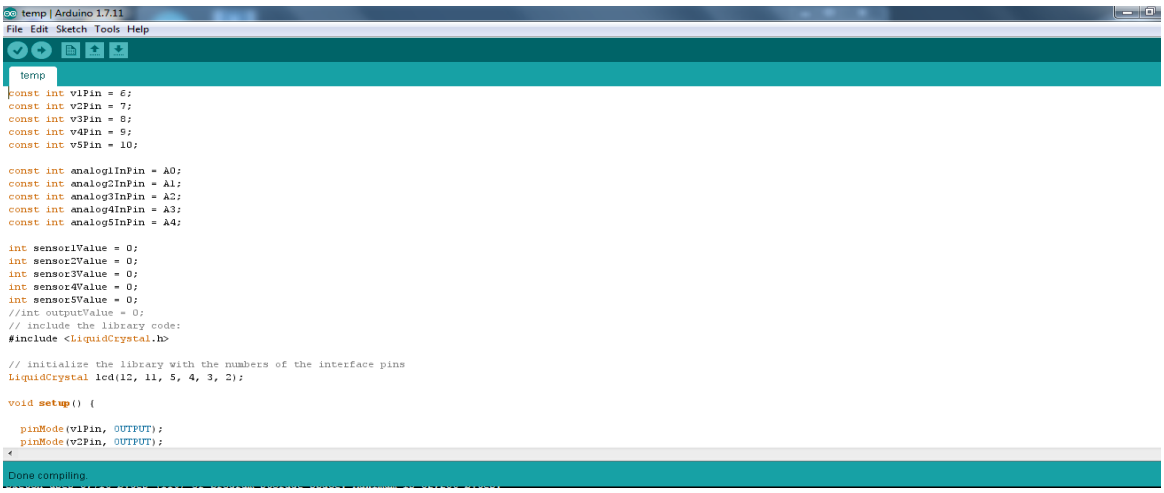


Fig:3.5.2.5: Upload the File

- *Upload Using Programmer*

  This will overwrite the bootloader on the board; you will need to use Tools > Burn Bootloader to restore it and be able to Upload to USB serial port again. However, it allows you to use the full capacity of the Flash memory for your sketch. Please note that this command will NOT burn the fuses. To do so a *Tools -> Burn Bootloader* command must be executed.

- *Export Compiled Binary*

  Saves a .hex file that may be kept as archive or sent to the board using other tools.
- *Show Sketch Folder*

  Opens the current sketch folder.
- *Include Library*

  Adds a library to your sketch by inserting #include statements at the start of your code. For more details, see libraries below. Additionally, from this menu item you can access the Library Manager and import new libraries from .zip files.
- *Add File...*

  Adds a source file to the sketch (it will be copied from its current location). The new file appears in a new tab in the sketch window. Files can be removed from the sketch using the tab menu accessible clicking on the small triangle icon below the serial monitor one on the right side o the toolbar.
-

3.5.3 TOOLS

- *Auto Format*

  This formats your code nicely: i.e. indents it so that opening and closing curly braces line up, and that the statements inside curly braces are indented more.
- *Archive Sketch*

  Archives a copy of the current sketch in .zip format. The archive is placed in the same directory as the sketch.
- *Fix Encoding & Reload*

  Fixes possible discrepancies between the editor char map encoding and other operating systems char maps.
- *Serial Monitor*

  Opens the serial monitor window and initiates the exchange of data with any connected board on the currently selected Port. This usually resets the board, if the board supports Reset over serial port opening.
- *Board*

  Select the board that you're using. See below for descriptions of the various boards.
- *Port*

  This menu contains all the serial devices (real or virtual) on your machine. It should automatically refresh every time you open the top-level tools menu.

- *Programmer*

  For selecting a harware programmer when programming a board or chip and not using the onboard USB-serial connection. Normally you won't need this, but if you're burning a bootloader to a new microcontroller, you will use this.

- *Burn Bootloader*

  The items in this menu allow you to burn a bootloader onto the microcontroller on an Arduino board. This is not required for normal use of an Arduino or Genuino board but is useful if you purchase a new ATmega microcontroller (which normally come without a bootloader). Ensure that you've selected the correct board from the Boards menu before burning the bootloader on the target board. This command also set the right fuses.

  Help

  Here you find easy access to a number of documents that come with the Arduino Software (IDE). You have access to Getting Started, Reference, this guide to the IDE and other documents locally, without an internet connection. The documents are a local copy of the online ones and may link back to our online website.

- *Find in Reference*

  This is the only interactive function of the Help menu: it directly selects the relevant page in the local copy of the Reference for the function or command under the cursor.

3.5.4 SKETCHBOOK

The Arduino Software (IDE) uses the concept of a sketchbook: a standard place to store your programs (or sketches). The sketches in your sketchbook can be opened from the File >Sketchbook menu or from the Open button on the toolbar. The first time you run the Arduino software, it will automatically create a directory for your sketchbook. You can view or change the location of the sketchbook location from with the Preferences dialog.

Beginning with version 1.0, files are saved with a .ino file extension. Previous versions use the. pde extension. You may still open .pde named files in version 1.0 and later, the software will automatically rename the extension to .ino.

Tabs, Multiple Files, and Compilation

Allows you to manage sketches with more than one file (each of which appears in its own tab). These can be normal Arduino code files (no visible extension), C files (.c extension), C++ files (.cpp), or header files (.h).

3.5.5UPLOADING

Before uploading your sketch, you need to select the correct items from the Tools > Board and Tools > Port menus. The boards are described below. On the Mac, the serial port is probably something like /dev/tty.usbmodem241 (for an Uno or Mega2560 or Leonardo) or /dev/tty.usbserial-1B1 (for a Duemilanove or earlier USB board), or /dev/tty.USA19QW1b1P1.1 (for a serial board connected with a Keyspan USB-to-Serial adapter). On Windows, it's probably COM1 or COM2 (for a serial board) or COM4, COM5, COM7, or higher (for a USB board) - to find out, you look for USB serial device in the ports section of the Windows Device Manager. On Linux, it should be /dev/ttyACMx , /dev/ttyUSBx or similar. Once you've selected the correct serial port and board, press the upload button in the toolbar or select the Upload item from the Sketch menu. Current Arduino boards will reset automatically and begin the upload. With older boards (pre-Diecimila) that lack auto-reset, you'll need to press the reset button on the board just before starting the upload. On most boards, you'll see the RX and TX LEDs blink as the sketch is uploaded. The Arduino Software (IDE) will display a message when the upload is complete, or show an error.

When you upload a sketch, you're using the Arduino bootloader, a small program that has been loaded on to the microcontroller on your board. It allows you to upload code without using any additional hardware. The bootloader is active for a few seconds when the board resets; then it starts whichever sketch was most recently uploaded to the microcontroller. The bootloader will blink the on-board (pin 13) LED when it starts (i.e. when the board resets).

LIBRARIES

Libraries provide extra functionality for use in sketches, e.g. working with hardware or manipulating data. To use a library in a sketch, select it from the Sketch > Import Library menu. This will insert one or more #include statements at the top of the sketch and compile the library with your sketch. Because libraries are uploaded to the board with your sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete its #includestatements from the top of your code.

There is a list of libraries in the reference. Some libraries are included with the Arduino software. Others can be downloaded from a variety of sources or through the Library Manager. Starting with version 1.0.5 of the IDE, you do can import a library from a zip file and use it in an open sketch. See these instructions for installing a third-party library.

To write your own library, see this tutorial.

Third-Party Hardware

Support for third-party hardware can be added to the hardware directory of your sketchbook directory. Platforms installed there may include board definitions (which appear in the board menu), core libraries, bootloaders, and programmer definitions. To install, create the hardware directory, then unzip the third-party platform into its own sub-directory. (Don't use "arduino" as the sub-directory name or you'll override the built-in Arduino platform.) To uninstall, simply delete its directory.

For details on creating packages for third-party hardware, see the Arduino IDE 1.5 3rd party Hardware specification.

SERIAL MONITOR

Displays serial data being sent from the Arduino or Genuino board (USB or serial board). To send data to the board, enter text and click on the "send" button or press enter. Choose the baud rate from the drop-down that matches the rate passed to Serial.begin in your sketch. Note that on Windows, Mac or Linux, the Arduino or Genuino board will reset (rerun your sketch execution to the beginning) when you connect with the serial monitor.

You can also talk to the board from Processing, Flash, MaxMSP, etc (see the interfacing page for details).

PREFERENCES

Some preferences can be set in the preferences dialog (found under the Arduino menu on the Mac, or File on Windows and Linux). The rest can be found in the preferences file, whose location is shown in the preference dialog.

LANGUAGE SUPPORT

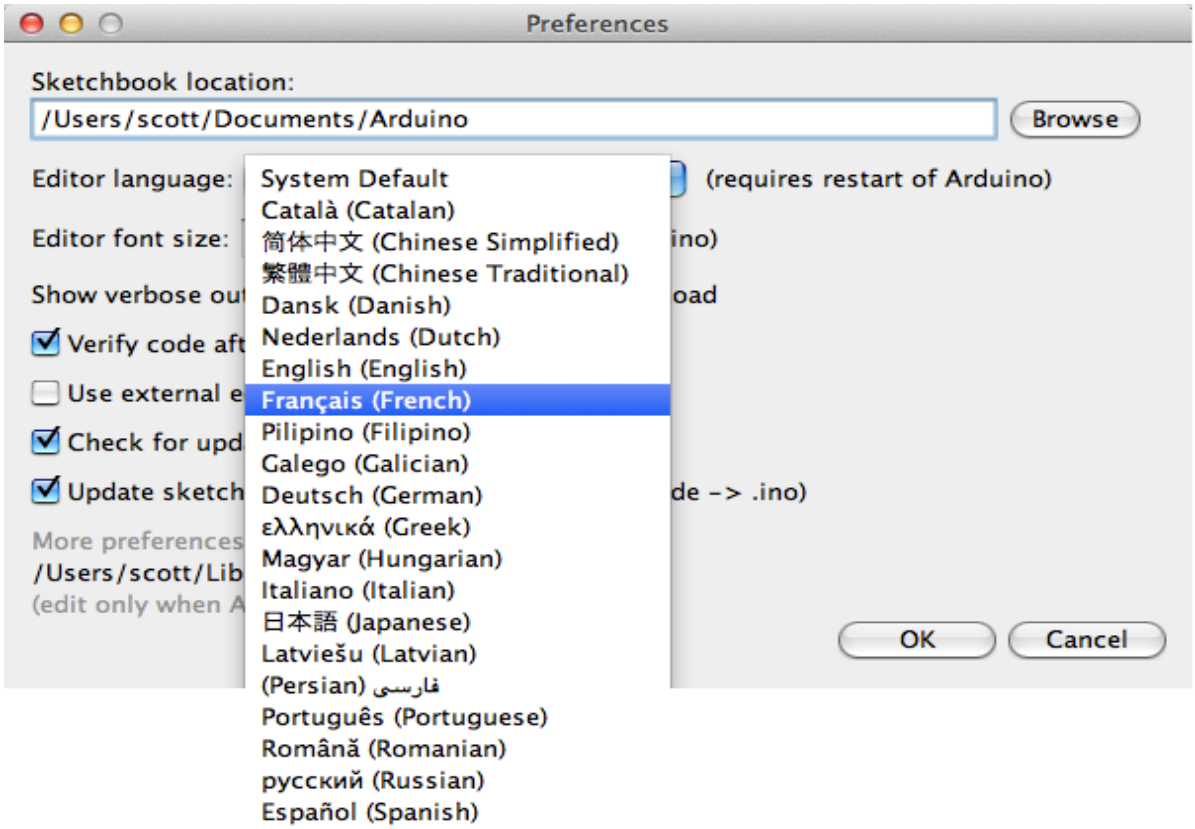


Fig:3.5.2.6: Language support

Since version 1.0.1 , the Arduino Software (IDE) has been translated into 30+ different languages. By default, the IDE loads in the language selected by your operating system. (Note: on Windows and possibly Linux, this is determined by the locale setting which controls currency and date formats, not by the language the operating system is displayed in.)

If you would like to change the language manually, start the Arduino Software (IDE) and open the Preferences window. Next to the Editor Language there is a dropdown menu of currently supported languages. Select your preferred language from the menu, and restart the software to use the selected language. If your operating system language is not supported, the Arduino Software (IDE) will default to English.

You can return the software to its default setting of selecting its language based on your operating system by selecting System Default from the Editor Language drop-down. This setting will take effect when you restart the Arduino Software (IDE). Similarly, after changing

your operating system's settings, you must restart the Arduino Software (IDE) to update it to the new default language.

BOARDS

The board selection has two effects: it sets the parameters (e.g. CPU speed and baud rate) used when compiling and uploading sketches; and sets and the file and fuse settings used by the burn bootloader command. Some of the board definitions differ only in the latter, so even if you've been uploading successfully with a particular selection, you'll want to check it before burning the bootloader. You can find a comparison table between the various boards here. Arduino Software (IDE) includes the built in support for the boards in the following list, all based on the AVR Core. The Boards Manager included in the standard installation allows to add support for the growing number of new boards based on different cores like Arduino Due, Arduino Zero, Edison, Galileo and so on.

3.6IMPLEMENTATION

CODE STRUCTURE

```
#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include "DHT.h"

#define SCREEN_WIDTH 128 // OLED display width, in pixels

#define SCREEN_HEIGHT 64 // OLED display height, in pixels

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

//SoftwareSerial = Serial2(8, 9);

#define DHTPIN 2

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

#define vib 5

#define fire 6
```

```
#define speaker0 A0
#define speaker1 A1
#define speaker2 A2
#define speaker3 A3
#define led 8
#define buzz 7
const int vib_State = 0;
const int fire_State = 0;
void setup(){
  // put your setup code here, to run once:
 Serial.begin(9600);
 dht.begin();
 // SSD1306_SWITCHCAPVCC = generate display voltage from 3.3V internally
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    //Serial.println(F("SSD1306 allocation failed"));
    while (true)
    ; // Don't proceed, loop forever
  }
  pinMode(vib,INPUT);
  pinMode(fire, INPUT);
  pinMode(speaker0, OUTPUT);
  pinMode(speaker1, OUTPUT);
  pinMode(speaker2, OUTPUT);
  pinMode(speaker3, OUTPUT);
  pinMode(led,OUTPUT);
```

```
 pinMode(buzz,OUTPUT);
 // wait for initializing
 display.clearDisplay();
 display.setTextSize(1);
 display.setTextColor(WHITE);
 display.setCursor(0, 0);
 display.println("INTELLIGENT EVENT DATA RECORDER FOR VEHICLES");
 display.display();
 delay(2000);

}
 void loop() {
 // put your main code here, to run repeatedly:
float h = dht.readHumidity();
float t = dht.readTemperature();
float f = dht.readTemperature(true);

 float hif = dht.computeHeatIndex(f, h);
 float hic = dht.computeHeatIndex(t, h, false);

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0,0);
display.print("Temperature :");
```

```
display.setTextSize(2);          // text size
display.setTextColor(WHITE);     // text color
 display.setCursor(0, 10);
 display.print(t);
 display.display();
 delay(500);
 if (t >= 45) {
   digitalWrite(buzz, HIGH);
   digitalWrite(speaker0,LOW);
   digitalWrite(speaker1,LOW);
   digitalWrite(speaker2,LOW);
   digitalWrite(speaker3,LOW);
   display.clearDisplay();       // clear display
   display.setTextSize(1);       // text size
   display.setTextColor(WHITE);  // text color
   display.setCursor(0, 0);      // position to display
   display.print("Temperature Abnormal!!!!");
   display.display();
   delay(800);
   digitalWrite(buzz, LOW);
  }

   display.setTextSize(1);       // text size
   display.setTextColor(WHITE);  // text color
   display.setCursor(0, 30);     // position to display
```

```
  display.print("Humidity :");  // text to display
  display.setTextSize(2);       // text size
  display.setTextColor(WHITE);  // text color
  display.setCursor(0, 40);
  display.print(h);
  display.display();
  delay(1500);
  if (h >= 80) {
    digitalWrite(buzz, HIGH);
    digitalWrite(speaker0,LOW);
    digitalWrite(speaker1,LOW);
    digitalWrite(speaker2,LOW);
    digitalWrite(speaker3,LOW);
    display.clearDisplay();       // clear display
    display.setTextSize(1);       // text size
    display.setTextColor(WHITE);  // text color
    display.setCursor(0, 0);      // position to display
    display.print("Humidity is Abnormal!!!");
    display.display();
    delay(800);
    digitalWrite(buzz, LOW);
    display.clearDisplay();
}
  display.clearDisplay();
  int vib_State = digitalRead(vib);
```

```
if (vib_State == HIGH)
{
  digitalWrite(buzz,HIGH);
  digitalWrite(speaker0,LOW);
  digitalWrite(speaker1,LOW);
  digitalWrite(speaker2,LOW);
  digitalWrite(speaker3,LOW);
  digitalWrite(led,HIGH);
  display.clearDisplay();
  display.setCursor(0, 20);
  display.setTextColor(WHITE);
  display.setTextSize(1);
  display.println("vibration Abnormal");
  display.display();
  delay(1000);
  }
else
{
  digitalWrite(buzz,LOW);
  digitalWrite(speaker0,HIGH);
  digitalWrite(speaker1,HIGH);
  digitalWrite(speaker2,HIGH);
  digitalWrite(speaker3,HIGH);
  digitalWrite(led,LOW);
```

```
  display.setCursor(0, 10);
  display.setTextColor(WHITE);
  display.setTextSize(1);
  display.println("vibration Normal");
  display.display();
  delay(1000);
  display.clearDisplay();
 }

int fire_State = digitalRead(fire);

if (fire_State == LOW)
{
  digitalWrite(buzz,HIGH);
  digitalWrite(speaker0,LOW);
  digitalWrite(speaker1,LOW);
 digitalWrite(speaker2,LOW);
  digitalWrite(speaker3,LOW);
  digitalWrite(led,HIGH);
  display.setCursor(0, 20);
  display.setTextColor(WHITE);
  display.setTextSize(1);
  display.println("fire Abnormal");
  display.display();
  delay(1000);
```

```
    }
  else
   {
    digitalWrite(buzz,LOW);
    digitalWrite(speaker0,HIGH);
    digitalWrite(speaker1,HIGH);
    digitalWrite(speaker2,HIGH);
    digitalWrite(speaker3,HIGH);
    digitalWrite(led,LOW);
    display.setCursor(0, 10);
    display.setTextColor(WHITE);
    display.setTextSize(1);
    display.println("fire Normal");
    display.display();
    delay(1000);
   }
}
```

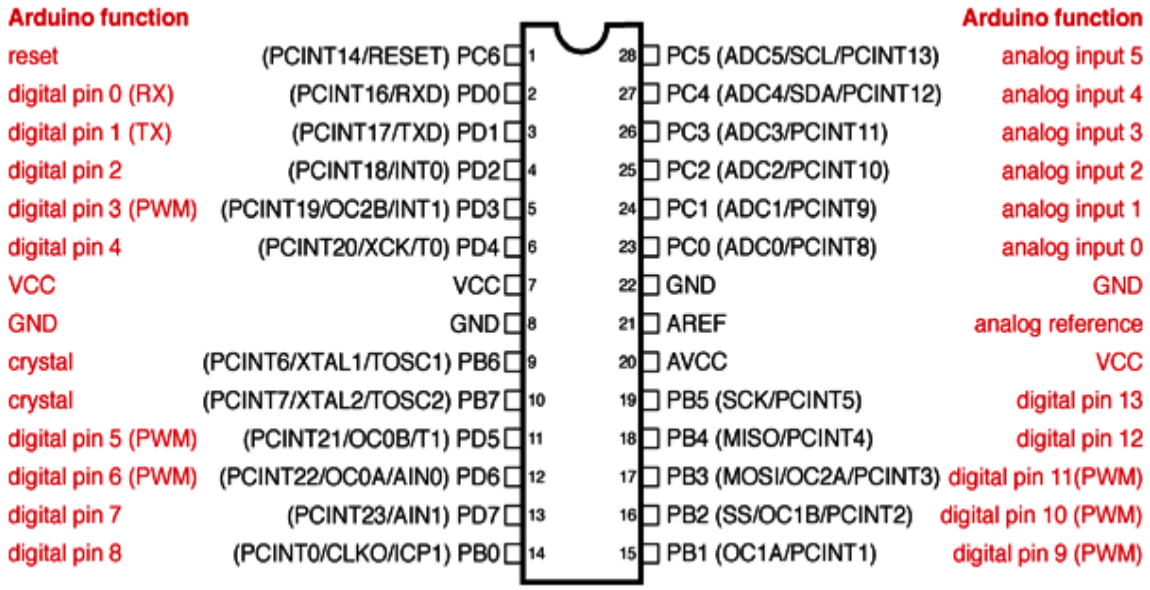


Fig: 3.6.1CIRCUIT PIN DIAGRAM

Each of the 14 digital pins on the uno can be used as an input or output, using pinmode(), digitalwrite (), and digitalread () functions. They operate at 5 volts. Each pin can provide or receive 20 ma as recommended operating condition and has an internal pull-up resistor (disconnected by default) of 20-50k ohm. A maximum of 40ma is the value that must not be exceeded on any i/o pin to avoid permanent damage to the microcontroller.

In addition, some pins have specialized functions:

Serial: 0 (rx) and 1 (tx). Used to receive (rx) and transmit (tx) ttl serial data. These pins are connected to the corresponding pins of the atmega8u2 usb-to-ttl serial chip.

External interrupts: 2 and 3. These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value. See the attachinterrupt() function for details.

Pwm: 3, 5, 6, 9, 10, and 11. Provide 8-bit pwm output with the analog write() function.

Spi: 10 (ss), 11 (mosi), 12 (miso), 13 (sck). These pins support spi communication using the spi library.

Led: 13. There is a built-in led driven by digital pin 13. When the pin is high value, the led is on, when the pin is low, it's off.

Twi: a4 or sda pin and a5 or scl pin. Support twi communication using the wire library.

The uno has 6 analog inputs, labeled a0 through a5, each of which provide 10 bits of resolution (i.e. 1024 different values). By default they measure from ground to 5 volts, though is it possible to change the upper end of their range using the aref pin and the analogreference() function.

There are a couple of other pins on the board:

Aref.Reference voltage for the analog inputs. Used with analogreference().

Reset. Bring this line low to reset the microcontroller. Typically used to add a reset button to shields which block the one on the board.

COMMUNICATION

Arduino/genuino uno has a number of facilities for communicating with a computer, another arduino/genuino board, or other microcontrollers. The atmega328 provides uartttl (5v) serial communication, which is available on digital pins 0 (rx) and 1 (tx). An atmega16u2 on the board channels this serial communication over usb and appears as a virtual com port to software on the computer. The 16u2 firmware uses the standard usb com drivers, and no external driver is needed. However, on windows, a .inf file is required. The arduino software (ide) includes a serial monitor which allows simple textual data to be sent to and from the board. The rx and txleds on the board will flash when data is being transmitted via the usb-to-serial chip and usb connection to the computer (but not for serial communication on pins 0 and 1).

A softwareserial library allows serial communication on any of the uno's digital pins.

The atmega328 also supports i2c (twi) and spi communication. The arduino software (ide) includes a wire library to simplify use of the i2c bus; see the documentation for details. For spi communication, use the spi library.

AUTOMATIC (SOFTWARE) RESET

Rather than requiring a physical press of the reset button before an upload, the arduino/genuino uno board is designed in a way that allows it to be reset by software running on a connected computer. One of the hardware flow control lines (dtr) of the atmega8u2/16u2 is connected to the reset line of the atmega328 via a 100 Nano farad capacitor. When this line is asserted (taken low), the reset line drops long enough to reset the chip. The arduino software (ide) uses this capability to allow you to upload code by simply pressing the upload button in

the interface toolbar. This means that the bootloader can have a shorter timeout, as the lowering of dtr can be well-coordinated with the start of the upload.

This setup has other implications. When the uno is connected to either a computer running mac os x or linux, it resets each time a connection is made to it from software (via usb). For the following half-second or so, the bootloader is running on the uno. While it is programmed to ignore malformed data (i.e. Anything besides an upload of new code), it will intercept the first few bytes of data sent to the board after a connection is opened. If a sketch running on the board receives one-time configuration or other data when it first starts, make sure that the software with which it communicates waits a second after opening the connection and before sending this data.

The uno board contains a trace that can be cut to disable the auto-reset. The pads on either side of the trace can be soldered together to re-enable it. It's labeled "reset-en". You may also be able to disable the auto-reset by connecting a 110 ohm resistor from 5v to the reset line; see this forum thread for details.

3.7 TESTING AND VALIDATION

1. Data Collection and Storage:EDRs collect data during and after accidents, recording information such as speed, steering wheel angle, yaw rate, and other vehicle dynamics. This data is stored in a memory chip, typically within the Airbag Control Module (ACM). The EDR must be tested to ensure reliable data collection and storage across various conditions.

2. Data Retrieval and Decoding: Data can be accessed through specialized software and tools, using a cable plugged into the vehicle's port or by retrieving the ACM module.Testing verifies the effectiveness of these retrieval methods and the ability to decode the data correctly.

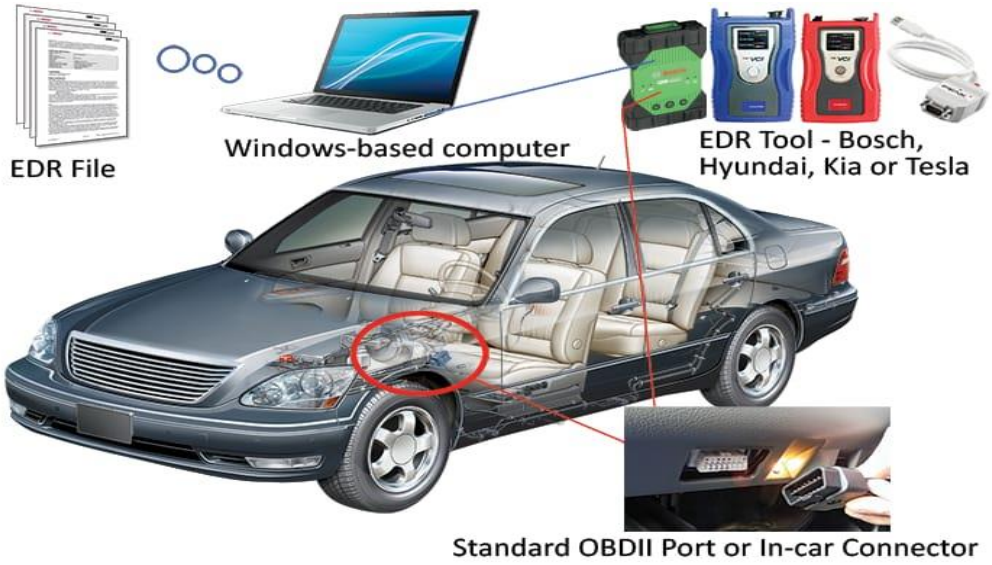
3. Data Validation:Comparing EDR data with data from independent measuring devices is crucial for validation. Testing involves comparing pre-crash and post-crash data from the EDR with reference data to ensure accuracy and reliability. Specific standards, like 49 CFR Part 563, define accuracy requirements for EDR data, such as the speed indicator accuracy.

4. Comprehensive Testing: Thorough testing should consider various crash scenarios and accident sequences to ensure the EDR functions as expected in different situations.

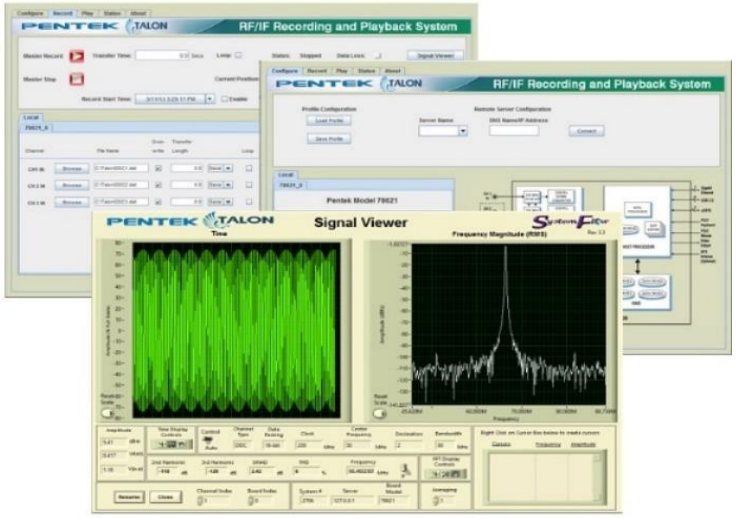
Testing principles like the "pesticide paradox" and "defects cluster together" can help in identifying potential issues and focusing on specific areas.

5. Usability and Forensic Application: The focus should also be on the usability of the CDR (Crash Data Retrieval) system and its outputs for forensic practice.This includes validating the ability to extract useful pre-crash and post-crash information for accident analysis.

3.8PHOTOGRAPHS



3.8.1fig: Hardware prototype



3.8.2 fig: Software GUIs

CHAPTER4

RESULTS AND DISCUSSION

Event Data Recorders (EDRs) in vehicles includes integrating with telematics and autonomous driving systems, enhancing data analysis through AI, and exploring new applications like driver education and augmented reality.

An Intelligent Event Data Recorder (I-EDR) is a system that uses event detectors and a Mealy machine to determine when to start and stop buffering data from various sources in a vehicle. This system focuses on capturing valuable data from both high and low-bandwidth sources, such as cameras, LiDARs, and the CAN bus. The data is then processed to compute its value based on the event's context, allowing for a more efficient and targeted data recording approach compared to traditional EDRS.

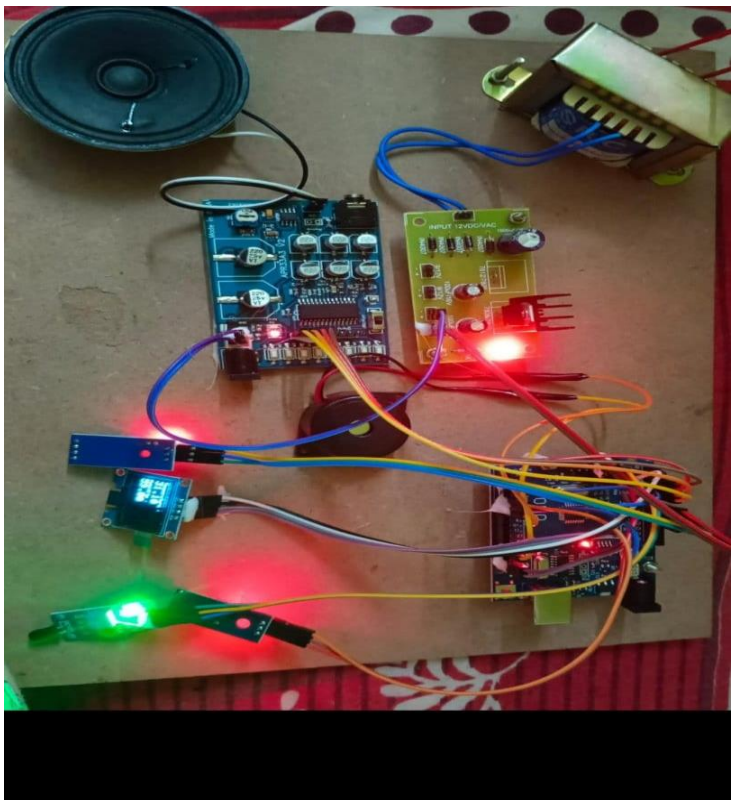


Fig:4.1:Intelligent event data recorder for vehicles

CHAPTER 5

CONCLUSION

The transport sector is in the process of being rapidly and fundamentally reshaped by new emerging technologies, including autonomous driving and collective intelligent transportation systems. This reshaping promises huge economic and social benefits yet brings equally huge challenges in terms of secure and safe technology development, its smooth integration to social fabric and the adaptation of legal and regulatory framework. In the previous article we have developed Policy Scan and Technology Strategy Design methodology [18] for identification of concrete societal expectations and problems and technological availabilities that can mitigate them in the domain of autonomous driving and smart mobility. As a result we have identified technology having a clear place in emerging technological and regulatory landscape as well as market deployment potential – Event Data Recorder for Autonomous Driving (EDR/AD). In this article, particularities of the context of EDR/AD related technology availabilities, regulatory initiatives and problematics of the large scale industry transition at European and international level are presented and discussed in depth. EDR/AD is one of the technological solutions which, if designed by integrating knowledge existing in the industry and academia, can help to design a robust and secure in-vehicle data recording, storage and access management solutions which will be a regulatory prerequisite for the deployment of autonomous vehicles. Transformation of transport sector involves many stakeholders and interest groups and it is important to understand that no one of them has a decisive power to shape the direction of the transformation (albeit some of them have more power than others). Guided by Policy Scan and Technology Strategy design methodology [18] we explicate ecosystem of actors (industry participants, policy makers, technology availabilities and more) around EDR/AD solution and its place in the C-ITS. The aim of this explication is to pave ground for collaboration of relevant stakeholders for developing and implementing concrete technology of EDR/AD, considering its short and long term effects on the transformation of smart mobility systems, regulatory aspects and business opportunities. Further steps for developing EDR/AD as envisioned in this paper involves building collaboration between vehicle manufacturers and security researchers in order to bring state of the art security technologies to the industry, proposing concrete use cases for prototyping and security analysis and developing a proof of comarket-deployment ready.

REFERENCES

[1] Article 29. Opinion 03/2017 on Processing personal dadta in the context of Cooperative Intelligent Transport Systems (C-ITS). Opinion. Article 29 Data Protection Working Party, 2017 (cit. on pp. 4 , 13 , 17).

[2] C-ITS Platform Working Group 6. Access to in-vehicle resources and data. Tech. rep. Dec. 2015 (cit. on p. 10).

[3] ACEA. Position Paper: Access to vehicle data for thir-party services. English. Position Paper. Brussels, Belgium: ACEA, Dec. 2016 (cit. on p. 11).

[4] European Association of Automotive Suppliers (CLEPA). Discussion Paper on Event Data Recorders for Automated Driving (EDR/AD). English. Sept. 2016 (cit. on pp. 13 –16).

[5] C-ITS Platformm WG5: Security & Certification. ANNEX 1 of Final Report: Trust models for Cooperative - Intelligent Transport System (C-ITS). English. Tech. rep. v1.1. C-ITS Platform, Jan. 2016 (cit. on p. 16).

[6] DG GROW. GEAR2030 Phase II Final Report - 2017. Tech. rep. European Comission, Oct. 2017 (cit. on pp. 5 , 6 , 13).

[7] C-ITS Platform Phase II. Certificate Policy for Deployment and Operation of European Cooperative Intelligent Transport Systems (C-ITS). Tech. rep. Release I. June 2017, p. 81 (cit. on p. 17).

[8] SAE International. “J3016A: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles -”. Sept. 2016 (cit. on p. 4).

[9] M McCarthy et al. Access to In-vehicle Data and Resources. Study comissioned by European Commission CPR2419. Brussels, May 2017 (cit. on pp. 10, 11).

[10] Jan Tobias Mühlberg, Job Noorman, and Frank Piessens. “Lightweight and Flexible Trust Assessment Modules for the Internet of Things”. In: Computer Security - ESORICS 2015 - 20th European Symposium on Research in Computer Security, Vienna, Austria, September 21-25, 2015, Proceedings, Part I. 2015, pp. 503–520 (cit. on p. 16).